MATLAB and Mathematical Experiments

Yifan Li

August 2026

Contents

Introduction **7**

1 MATLAB Basics and Common Commands **8**
1.1 MATLAB as a Computational Environment 9
1.2 Variables and Basic Data Types 9
1.3 Vectors and Matrices 10
1.4 Matrix Indexing 10
1.5 Matrix and Element-Wise Operations 11
1.6 Common Mathematical Functions 11
1.7 Basic Linear Algebra Commands 12
1.8 Strings and Character Arrays 12
1.9 Input, Output, and Display 13
1.10 Useful Array Manipulation 13
1.11 Basic Plotting 14
1.12 Chapter Shorthand 15

2 Control Flow and Program Structure **17**
2.1 Logical Expressions 18
2.2 Conditional Statements 18
2.3 The switch Statement 19
2.4 The for Loop 20
2.5 Nested Loops 20
2.6 The while Loop 21
2.7 Stopping Criteria 21
2.8 break and continue 22
2.9 Finding Entries by Logical Conditions 22
2.10 Scripts and Functions 22
2.11 Anonymous Functions 23
2.12 A Standard Iterative Program 23
2.13 Chapter Shorthand 24

3 Visualization and Experimental Mathematics **26**
3.1 Visualization as a Mathematical Experiment 27
3.2 Basic Two-Dimensional Plots 27
3.3 Plot Appearance and Coordinate Control 28
3.4 Multiple Figures and Subplots 28
3.5 Function Plotting and Coordinate Transformations 29
3.6 Scatter Plots and Data Visualization 29
3.7 Polar and Parametric Curves 29
3.8 Surfaces and Mesh Grids 30
3.9 Contour Plots 31
3.10 Vector Fields and Motion 31
3.11 Simple Animation 32
3.12 Chapter Shorthand 32

4 Sequences, Series, and Experimental Exploration **34**
4.1 The Experimental Method 35
4.2 Linear Periodic Functions 35

4.3 Antiderivatives of Periodic Functions . . . 36
4.4 Fibonacci Sequence and Exponential Growth . . . 36
4.5 Harmonic Series and Growth Rate . . . 37
4.6 Exploring General Sequences and Series . . . 38
4.7 Linearization as an Experimental Tool . . . 39
4.8 Chapter Shorthand . . . 39

5 Matrix Experiments and Linear Algebra . . . 41
5.1 Matrices as Mathematical Models . . . 42
5.2 Linear Iteration . . . 42
5.3 Eigenvalues and Dominant Directions . . . 42
5.4 Diagonalization . . . 43
5.5 The Power Method . . . 43
5.6 Convergence of the Power Method . . . 44
5.7 Inverse Power Method . . . 45
5.8 Transition Matrices and Long-Term Behavior . . . 45
5.9 Matrices and Graphs . . . 46
5.10 Permutation Parity and the Fifteen Puzzle . . . 46
5.11 Chapter Shorthand . . . 47

6 Stochastic Simulation and Monte Carlo Methods . . . 49
6.1 Random Simulation . . . 50
6.2 Basic Random Number Generation . . . 50
6.3 Random Seeds and Reproducibility . . . 51
6.4 Simulating Common Distributions . . . 51
6.5 Frequency Approximation of Probability . . . 51
6.6 Geometric Probability: The Meeting Problem . . . 52
6.7 Simulation of Sequential Random Processes . . . 53
6.8 Sampling Without Replacement . . . 53
6.9 Monte Carlo Estimation of Expectations . . . 53
6.10 Simulation-Based Decision Making . . . 54
6.11 Monte Carlo for Testing Conjectures . . . 55
6.12 Accuracy and Sample Size . . . 55
6.13 Chapter Shorthand . . . 56

7 Computing π and Convergence Experiments . . . 58
7.1 One Quantity, Different Algorithms . . . 59
7.2 Taylor Series for the Arctangent . . . 59
7.3 Why the Direct Series Converges Slowly . . . 59
7.4 A Faster Arctangent Decomposition . . . 60
7.5 Machin's Formula . . . 61
7.6 Error and Convergence Experiments . . . 61
7.7 Monte Carlo Estimation of π . . . 62
7.8 Accuracy of the Monte Carlo Method . . . 62
7.9 Why Monte Carlo Is Still Useful . . . 63
7.10 Buffon's Needle Experiment . . . 63
7.11 Deterministic and Stochastic Methods . . . 64
7.12 Chapter Shorthand . . . 65

8 Symbolic Computation . . . 67

8.1 Numerical and Symbolic Computation . . . 68
8.2 Symbolic Variables and Expressions . . . 68
8.3 Substitution . . . 68
8.4 Simplification and Algebraic Manipulation . . . 69
8.5 Symbolic Functions . . . 69
8.6 Exact and Arbitrary-Precision Values . . . 70
8.7 Limits, Sums, and Products . . . 70
8.8 Symbolic Differentiation . . . 70
8.9 Taylor Expansion . . . 71

9 Nonlinear Equations and Iterative Methods . . . 72
9.1 Numerical Root Finding . . . 73
9.2 Bisection Method . . . 73
9.3 Fixed-Point Iteration . . . 74
9.4 Convergence of Fixed-Point Iteration . . . 74
9.5 Cobweb Diagrams . . . 75
9.6 Newton's Method . . . 75
9.7 Limitations of Newton's Method . . . 76
9.8 Secant Method . . . 76
9.9 Nonlinear Systems . . . 77
9.10 MATLAB Built-In Root Solvers . . . 78
9.11 Polynomial Roots . . . 78
9.12 Visualization and Initial Values . . . 78
9.13 Iteration and Complex Behavior . . . 79
9.14 Chapter Shorthand . . . 79

10 Interpolation and Data Fitting . . . 81
10.1 Interpolation and Fitting . . . 82
10.2 Polynomial Interpolation . . . 82
10.3 Piecewise Linear Interpolation . . . 83
10.4 Cubic Spline Interpolation . . . 83
10.5 Interpolation Versus Smoothing . . . 84
10.6 Two-Dimensional Interpolation . . . 84
10.7 Scattered Data Interpolation . . . 85
10.8 Least-Squares Fitting . . . 85
10.9 Polynomial Fitting . . . 86
10.10 Linearization of Nonlinear Relations . . . 87
10.11 Nonlinear Least-Squares Fitting . . . 87
10.12 Residual-Based Nonlinear Least Squares . . . 88
10.13 Constrained Least Squares . . . 88
10.14 Model Selection from Data . . . 89
10.15 Application Pattern: Drug Concentration . . . 89
10.16 Chapter Shorthand . . . 89

11 Numerical Integration . . . 92
11.1 From Definite Integrals to Numerical Quadrature . . . 93
11.2 Rectangle Rules . . . 93
11.3 Composite Trapezoidal Rule . . . 94
11.4 Composite Simpson Rule . . . 94

11.5 Interpolation Viewpoint . . . 95
11.6 Convergence Experiments . . . 95
11.7 The `trapz` Function . . . 96
11.8 Adaptive Quadrature . . . 96
11.9 The `quad` Function . . . 96
11.10The `quadl` Function . . . 97
11.11The `quadgk` Function . . . 97
11.12Double Integrals . . . 97
11.13Nonrectangular Regions . . . 98
11.14MATLAB Multidimensional Quadrature . . . 98
11.15Choosing a Numerical Integration Method . . . 99
11.16Chapter Shorthand . . . 99

12 Optimization . . . 101
12.1 Optimization Models . . . 102
12.2 Local and Global Optima . . . 102
12.3 First-Order Optimality . . . 102
12.4 Gradient Descent . . . 103
12.5 One-Dimensional Unconstrained Optimization . . . 104
12.6 Unconstrained Multivariable Optimization . . . 104
12.7 Linear Programming . . . 105
12.8 Linear Programming Modeling . . . 105
12.9 Quadratic Programming . . . 106
12.10General Nonlinear Programming . . . 106
12.11Nonlinear Constraint Functions . . . 107
12.12Initial Values and Local Minima . . . 108
12.13Integer Programming . . . 108
12.14Heuristic and Intelligent Optimization . . . 108
12.15Choosing an Optimization Solver . . . 109
12.16Chapter Shorthand . . . 109

13 Differential Equations . . . 111
13.1 Differential Equations as Dynamic Models . . . 112
13.2 Forward Euler Method . . . 112
13.3 Backward Euler Method . . . 113
13.4 Trapezoidal and Crank–Nicolson Method . . . 113
13.5 Improved Euler Method . . . 114
13.6 Runge–Kutta Methods . . . 114
13.7 Classical Fourth-Order Runge–Kutta Method . . . 114
13.8 Comparing Numerical Methods . . . 115
13.9 From Higher-Order Equations to Systems . . . 116
13.10MATLAB ODE Solvers . . . 117
13.11Using `ode45` . . . 117
13.12Systems with `ode45` . . . 117
13.13Phase-Plane Visualization . . . 118
13.14Parameters in ODE Models . . . 118
13.15Application Pattern: Pursuit Model . . . 119
13.16Online and Offline Computation . . . 119
13.17Choosing an ODE Solver . . . 119

13.18Chapter Shorthand 120

Acknowledgements **122**

Introduction

MATLAB is both a programming language for numerical computation and a practical environment for mathematical experimentation. In many mathematical problems, obtaining an exact symbolic solution is either impossible or unnecessary. Instead, computation can be used to explore examples, test conjectures, approximate solutions, visualize structures, and compare the behavior of different numerical methods.

The purpose of these notes is therefore twofold. The first part develops the basic MATLAB language needed for mathematical computation. It begins with frequently used commands, variables, vectors and matrices, indexing, strings, built-in functions, input and output, and other elementary syntax. It then introduces logical expressions, conditional statements, loops, termination conditions, and basic program organization. These chapters are intended mainly as a compact MATLAB reference rather than a complete programming manual.

The second and main part concerns *mathematical experiments*. A typical mathematical experiment starts from a mathematical problem and replaces part of the purely analytic reasoning by a reproducible computational procedure. The general workflow can be summarized as

$$\begin{gathered}\text{mathematical problem} \longrightarrow \text{mathematical or numerical formulation} \\ \text{algorithm} \longrightarrow \text{MATLAB implementation} \longrightarrow \text{numerical observation.}\end{gathered}$$

The computational result is not merely an output of a program. It should be interpreted mathematically: one should understand what is being approximated, which assumptions are involved, how numerical errors may arise, and whether the observed behavior is consistent with the underlying theory.

Accordingly, each chapter is organized around the essential mathematical ideas rather than around isolated MATLAB commands. Definitions, models, algorithms, and important formulas are introduced first, with one or two representative examples when they help clarify the method. Detailed numerical calculations are generally omitted. The corresponding MATLAB syntax and frequently reusable code patterns are then collected as a compact implementation reference.

A central principle throughout these notes is that computation complements mathematical reasoning rather than replacing it. Numerical experiments can suggest patterns, reveal exceptional cases, compare competing methods, and provide evidence for a conjecture, but the meaning and reliability of such evidence must still be justified mathematically. Conversely, mathematical analysis explains why an algorithm works and what its limitations are, while MATLAB makes these ideas directly testable.

Thus, these notes may be viewed as a concise bridge between mathematical theory and computational practice: MATLAB provides the language, algorithms provide the mechanism, and mathematical experiments provide the methodology.

Chapter 1

MATLAB Basics and Common Commands

1.1 MATLAB as a Computational Environment

MATLAB is a numerical computing environment designed primarily for matrix-based computation. Most mathematical objects in MATLAB are represented as arrays, so vectors and matrices form the basic language of computation.
A command is executed after pressing Enter. A semicolon suppresses the output, while omitting it displays the result in the Command Window.

```
a = 3
b = 5;
c = a + b;
```

Several commands are useful for managing the working environment.

```
clc                    % Clear Command Window
clear                  % Remove all variables
clear x                % Remove variable x
close all              % Close all figure windows
who                    % List current variables
whos                   % Detailed variable information
pwd                    % Current working directory
cd foldername          % Change working directory
```

The command `help` gives a short description of a function, while `doc` opens the corresponding MATLAB documentation.

```
help eig
doc eig
```

1.2 Variables and Basic Data Types

MATLAB does not require explicit declaration of ordinary variables. A variable is created when a value is assigned to it.
Variable names are case-sensitive and must begin with a letter. MATLAB also provides several predefined constants and special values.

```
x = 2.5;
n = 10;
z = 3 + 4i;

pi                     % Pi
Inf                    % Infinity
NaN                    % Not-a-number
eps                    % Floating-point relative precision
i                      % Imaginary unit
j                      % Imaginary unit
```

The command `class` identifies the data type of an object, and `size`, `length`, and `numel` describe its dimensions.

```
class(x)
size(A)
length(v)
numel(A)
```

1.3 Vectors and Matrices

A row vector is constructed by separating entries with spaces or commas. A column vector uses semicolons. Matrices are constructed row by row.

```
u = [1 2 3 4];
v = [1; 2; 3; 4];

A = [1 2 3;
     4 5 6;
     7 8 9];
```

The colon operator is one of the most frequently used MATLAB constructions. The expression `a:h:b` generates values from a to b with step size h. If the step size is omitted, MATLAB uses $h = 1$.

```
x = 1:5;
x = 0:0.1:1;

x = linspace(0,1,11);
y = logspace(0,3,4);
```

Frequently used special matrices include zero matrices, matrices of ones, identity matrices, and random matrices.

```
zeros(3,4)
ones(3,4)
eye(4)

rand(3,4)            % Uniform random entries
randn(3,4)           % Standard normal entries
```

1.4 Matrix Indexing

MATLAB indices begin at 1. For a matrix A, the expression `A(i,j)` denotes the entry in row i and column j.
The colon symbol selects an entire row or column, while `end` refers to the final valid index of the corresponding dimension.

```
A(2,3)               % Entry in row 2, column 3
A(2,:)               % Entire second row
A(:,3)               % Entire third column

A(1:2,2:4)           % Submatrix
A(end,:)             % Last row
A(:,end)             % Last column
A(2:end,:)           % Rows 2 through the last row
```

A single index accesses matrix entries using column-major order.

```
A(1)
A(2)
A(:)                 % Convert A into one column vector
```

Entries may also be modified or deleted through indexing.

```
A(2,3) = 10;
A(:,2) = 0;
A(:,3) = [];
```

1.5 Matrix and Element-Wise Operations

MATLAB distinguishes *matrix operations* from *element-wise operations*. This distinction is essential in numerical computation.
For compatible matrices, `*`, `/`, and `^` represent matrix operations. Adding a dot produces the corresponding element-wise operation.

```
A + B
A - B
A * B                % Matrix multiplication
A^2                   % Matrix power

A .* B               % Element-wise multiplication
A ./ B               % Element-wise division
A .^ 2               % Square each entry
```

For example, if a vector x represents sampling points and we want to evaluate $f(x) = x^2 + 2x + 1$ at every point, element-wise powers are required.

```
x = 0:0.1:2;
y = x.^2 + 2*x + 1;
```

The transpose operators are also frequently used.

```
A'                    % Conjugate transpose
A.'                   % Transpose without conjugation
```

1.6 Common Mathematical Functions

Most elementary mathematical functions operate element-wise when applied to vectors or matrices.

```
sqrt(x)
abs(x)
exp(x)
log(x)
log10(x)

sin(x)
cos(x)
tan(x)

asin(x)
acos(x)
atan(x)

floor(x)
ceil(x)
round(x)
```

```
fix(x)
```

Common functions for summarizing numerical data include the following.

```
sum(x)
prod(x)
max(x)
min(x)
mean(x)
median(x)
std(x)
var(x)

sort(x)
cumsum(x)
cumprod(x)
```

For matrices, many of these functions operate column by column by default.

```
sum(A)
sum(A,1)              % Sum along rows: one value per column
sum(A,2)              % Sum along columns: one value per row

max(A)
mean(A,2)
```

1.7 Basic Linear Algebra Commands

Since MATLAB is matrix-oriented, standard linear algebra operations are directly available.

```
det(A)                % Determinant
rank(A)               % Rank
trace(A)              % Trace
inv(A)                % Matrix inverse
norm(A)               % Matrix or vector norm

eig(A)                % Eigenvalues
[V,D] = eig(A)        % Eigenvectors and eigenvalues

rref(A)               % Reduced row echelon form
```

For a linear system $Ax = b$, the preferred MATLAB syntax is the backslash operator rather than explicitly computing A^{-1}.

```
x = A\b;
```

In general, `A\b` is more efficient and numerically more stable than `inv(A)*b`.

1.8 Strings and Character Arrays

MATLAB distinguishes character arrays, written with single quotation marks, from string objects, written with double quotation marks.

```
c = 'MATLAB';         % Character array
```

```
s = "MATLAB";          % String

length(c)
strlength(s)
```

Character arrays can be concatenated using brackets, while strings can be combined using the + operator.

```
a = 'Math';
b = 'Lab';
c = [a ' ' b];

s1 = "Mathematical";
s2 = "Experiment";
s = s1 + " " + s2;
```

Frequently used string operations include conversion, comparison, searching, and replacement.

```
num2str(3.14)
str2double("3.14")

strcmp('abc','abc')
contains("MATLAB experiment","MATLAB")
replace("a-b-c","-","+")
```

1.9 Input, Output, and Display

The function `input` reads information from the Command Window. Formatted output is usually produced using `fprintf`.

```
x = input('Enter x: ');

name = input('Enter name: ','s');

fprintf('x = %.4f\n',x);
fprintf('n = %d\n',n);
fprintf('x = %.3e\n',x);
```

Common format specifiers include `%d` for integers, `%f` for floating-point numbers, `%e` for scientific notation, and `%s` for strings or character arrays.
MATLAB's display precision can also be changed without modifying the stored numerical value.

```
format short
format long
format short e
format long e
format rat
```

1.10 Useful Array Manipulation

Reshaping and combining arrays occur frequently in mathematical experiments.

```
reshape(A,2,6)
A(:)

[A B]                   % Horizontal concatenation
[A; B]                  % Vertical concatenation

diag(v)                 % Vector to diagonal matrix
diag(A)                 % Extract diagonal

triu(A)                 % Upper triangular part
tril(A)                 % Lower triangular part
```

The dimensions of arrays can be inspected before performing operations.

```
[m,n] = size(A);
n = length(v);
N = numel(A);
```

1.11 Basic Plotting

Visualization is a central component of mathematical experiments. For sampled points (x_i, y_i), the command `plot(x,y)` produces a two-dimensional curve.

```
x = linspace(0,2*pi,200);
y = sin(x);

plot(x,y)
xlabel('x')
ylabel('sin(x)')
title('Graph of sin(x)')
grid on
```

Several curves may be drawn in the same figure.

```
plot(x,sin(x))
hold on
plot(x,cos(x))
hold off

legend('sin(x)','cos(x)')
```

Frequently used visualization commands include:

```
figure
subplot(2,1,1)
plot(x,y)

scatter(x,y)
bar(x,y)
histogram(x)

xlim([0 10])
ylim([-1 1])
grid on
```

At this stage, plotting is treated only as basic MATLAB syntax. Later mathematical experiments will use visualization to examine numerical solutions, approximation errors, convergence, and qualitative mathematical behavior.

1.12 Chapter Shorthand

The following collection summarizes the commands that are most frequently reused in later mathematical experiments.

```
% Environment
clc; clear; close all;

% Vectors and sampling
x = a:h:b;
x = linspace(a,b,n);

% Matrices
A = [1 2; 3 4];
zeros(m,n);
ones(m,n);
eye(n);

% Indexing
A(i,j);
A(i,:);
A(:,j);
A(:);

% Element-wise computation
y = x.^2;
z = x.*y;
q = x./y;

% Basic statistics
sum(x);
mean(x);
max(x);
min(x);
std(x);

% Linear algebra
rank(A);
det(A);
eig(A);
x = A\b;

% Output
fprintf('x = %.4f\n',x);

% Plot
plot(x,y);
xlabel('x');
```

```
44 ylabel('y');
45 grid on;
```

The essential MATLAB habit is to think in terms of arrays rather than individual scalar operations. Whenever possible, vectorized expressions should be preferred to repeated scalar calculations. Control structures and explicit iteration are introduced in the next chapter.

Chapter 2

Control Flow and Program Structure

2.1 Logical Expressions

Control flow is based on logical expressions. MATLAB represents a true statement by logical 1 and a false statement by logical 0.
The basic relational operators are:

```
a == b                  % Equal
a ~= b                  % Not equal
a < b
a <= b
a > b
a >= b
```

Logical operators combine or modify logical expressions.

```
A & B                   % Element-wise AND
A | B                   % Element-wise OR
~A                      % NOT

A && B                  % Short-circuit AND
A || B                  % Short-circuit OR
```

The operators `&` and `|` act element-wise and therefore naturally apply to arrays. The operators `&&` and `||` are used for scalar logical conditions and may avoid evaluating the second expression when its value is unnecessary.

```
x = [-2 0 1 3];

x > 0
(x >= 0) & (x <= 2)

a = 3;
b = 5;
(a > 0) && (b > a)
```

Several built-in functions are useful when a logical array must be reduced to a single logical value.

```
any(x > 0)              % At least one entry satisfies condition
all(x > 0)              % Every entry satisfies condition
```

2.2 Conditional Statements

The `if` statement executes a block of commands only when a condition is true.

```
if x > 0
    y = sqrt(x);
end
```

Multiple cases can be handled using `elseif` and `else`.

```
if x > 0
    s = 1;
elseif x == 0
    s = 0;
else
```

```
    s = -1;
end
```

For example, a piecewise function

$$f(x) = \begin{cases} x^2, & x \geq 0, \\ -x, & x < 0 \end{cases}$$

can be implemented for a scalar input as follows.

```
if x >= 0
    y = x^2;
else
    y = -x;
end
```

Nested conditional statements are possible, although excessive nesting usually makes a program difficult to read.

```
if x >= 0
    if x <= 1
        y = x^2;
    else
        y = x;
    end
else
    y = 0;
end
```

2.3 The switch Statement

When one variable is compared against several discrete possibilities, `switch` may be clearer than a long sequence of `if-elseif` statements.

```
switch method
    case 'Euler'
        order = 1;
    case 'Trapezoidal'
        order = 2;
    otherwise
        order = 0;
end
```

Several values can correspond to the same case.

```
switch n
    case {1,3,5,7,9}
        type = 'odd';
    case {2,4,6,8,10}
        type = 'even';
    otherwise
        type = 'outside range';
end
```

2.4 The for Loop

A `for` loop repeats a collection of commands over a prescribed sequence of values. Its basic structure is:

```
for k = 1:n
    statements
end
```

For example, the sum

$$S = \sum_{k=1}^{n} k^2$$

can be computed iteratively.

```
S = 0;

for k = 1:n
    S = S + k^2;
end
```

The loop variable does not have to increase by 1.

```
for x = 0:0.1:1
    y = exp(-x^2);
end
```

When the number of repetitions is known in advance, a `for` loop is usually the natural control structure.

2.5 Nested Loops

Loops may be nested when computation involves multiple indices. This occurs frequently in matrix construction and discrete mathematical experiments.
For example, the matrix defined by $A_{ij} = i + j$ can be generated by:

```
A = zeros(m,n);

for i = 1:m
    for j = 1:n
        A(i,j) = i + j;
    end
end
```

Preallocating the matrix with `zeros` is generally preferable to enlarging it repeatedly inside the loop.
For many simple array operations, explicit loops can be replaced by vectorized MATLAB expressions. For example,

```
for k = 1:n
    y(k) = x(k)^2;
end
```

can usually be replaced by:

```
y = x.^2;
```

Vectorization often produces shorter and clearer numerical code.

2.6 The while Loop

A `while` loop repeats commands as long as a logical condition remains true.

```
while condition
    statements
end
```

It is especially useful when the number of iterations is not known in advance. For example, an iterative process may stop once the error is smaller than a prescribed tolerance ε.

```
err = Inf;
k = 0;

while err > tol
    xnew = iteration(x);
    err = abs(xnew - x);
    x = xnew;
    k = k + 1;
end
```

Thus, `for` is naturally associated with a predetermined number of iterations, whereas `while` is naturally associated with a stopping criterion.

2.7 Stopping Criteria

Many numerical experiments generate a sequence $x_0, x_1, x_2, \ldots$ and terminate when successive approximations are sufficiently close.
An absolute stopping criterion has the form

$$|x_{k+1} - x_k| < \varepsilon,$$

while a relative stopping criterion may use

$$\frac{|x_{k+1} - x_k|}{\max(1, |x_{k+1}|)} < \varepsilon.$$

A maximum number of iterations should usually be included to prevent an unexpected infinite loop.

```
tol = 1e-8;
maxIter = 1000;
err = Inf;
k = 0;

while err > tol && k < maxIter
    xnew = iteration(x);
    err = abs(xnew - x);
    x = xnew;
    k = k + 1;
end
```

A typical numerical iteration therefore contains three components: an update rule, an error measure, and a termination condition.

2.8 break and continue

The command `break` immediately terminates the innermost loop.

```
for k = 1:1000
    x = update(x);

    if abs(x - target) < tol
        break
    end
end
```

The command `continue` skips the remainder of the current iteration and moves directly to the next iteration.

```
for k = 1:n
    if x(k) < 0
        continue
    end

    y(k) = sqrt(x(k));
end
```

Thus, `break` terminates a loop, whereas `continue` terminates only the current iteration.

2.9 Finding Entries by Logical Conditions

In MATLAB, many simple conditional operations can be performed without explicit loops. A logical condition applied to an array produces a logical array of the same size.

```
x = [-3 -1 0 2 5];

I = x > 0;
y = x(I);
```

Logical indexing can also modify selected entries directly.

```
x(x < 0) = 0;
```

The function `find` returns the numerical indices at which a logical condition is satisfied.

```
idx = find(x > 0);
```

For matrices, row and column indices can be obtained separately.

```
[row,col] = find(A > 10);
```

Logical indexing is one of the most useful MATLAB techniques and often avoids unnecessary conditional loops.

2.10 Scripts and Functions

A *script* is a MATLAB file containing a sequence of commands. Scripts operate in the current workspace and are useful for organizing complete mathematical experiments.
A simple script may have the form:

```
clc;
clear;
```

```
close all;

n = 100;
x = linspace(0,1,n);
y = exp(-x.^2);

plot(x,y);
grid on;
```

A *function* receives input arguments and returns output arguments. Its basic form is:

```
function y = f(x)
    y = x.^2 + 1;
end
```

Several outputs can be returned simultaneously.

```
function [s,p] = sumprod(a,b)
    s = a + b;
    p = a * b;
end
```

The function can then be called by:

```
[s,p] = sumprod(3,4);
```

Functions have their own local workspace, which helps separate a numerical method from the script that performs the experiment.

2.11 Anonymous Functions

A short mathematical function can be represented by an anonymous function without creating a separate function file.
For example, $f(x) = e^{-x^2} \sin x$ can be defined by:

```
f = @(x) exp(-x.^2).*sin(x);

y = f(1);
x = linspace(0,5,100);
plot(x,f(x));
```

Functions of several variables are defined similarly.

```
f = @(x,y) x.^2 + y.^2;
z = f(2,3);
```

Anonymous functions are particularly useful because many numerical MATLAB functions accept another function as an input. They will therefore appear frequently in later mathematical experiments.

2.12 A Standard Iterative Program

A large class of numerical algorithms can be expressed through the same basic program structure. Starting from x_0, compute

$$x_{k+1} = F(x_k)$$

until a prescribed stopping criterion is satisfied.
A reusable MATLAB structure is:

```
x = x0;
tol = 1e-8;
maxIter = 1000;

for k = 1:maxIter
    xnew = F(x);
    err = abs(xnew - x);

    if err < tol
        break
    end

    x = xnew;
end
```

For vector-valued iterations, the scalar absolute value is usually replaced by a norm.

```
err = norm(xnew - x);
```

This pattern—initialization, iteration, error measurement, and termination—is fundamental in numerical computation and will reappear in root finding, optimization, numerical differential equations, and many other mathematical experiments.

2.13 Chapter Shorthand

The essential control-flow syntax can be summarized as follows.

```
% Logical conditions
x == y;
x ~= y;
x > y;
x >= y;

A & B;
A | B;
~A;

any(A);
all(A);

% Conditional statement
if condition
    statements
elseif condition
    statements
else
    statements
end

% For loop
for k = 1:n
```

```
    statements
end

% While loop
while condition
    statements
end

% Loop control
break
continue

% Logical indexing
x(x < 0) = 0;
idx = find(x > 0);

% Anonymous function
f = @(x) exp(-x.^2).*sin(x);

% Function
function y = f(x)
    y = x.^2 + 1;
end
```

The central programming principle is to choose the simplest structure that expresses the mathematical procedure. Array operations and logical indexing should be used when the computation is naturally vectorized; `for` loops are appropriate for predetermined iterations; and `while` loops are appropriate when termination is determined dynamically. From the next chapter onward, these MATLAB tools will be used to construct actual mathematical experiments.

Chapter 3

Visualization and Experimental Mathematics

3.1 Visualization as a Mathematical Experiment

Visualization is one of the simplest forms of mathematical experimentation. For complicated functions or large data sets, a graph often reveals properties that are difficult to recognize directly from formulas or tables. It can suggest periodicity, monotonicity, oscillation, asymptotic behavior, symmetry, singularities, and geometric structure. This is the central role of the first mathematical experiment in the course. :contentReference[oaicite:0]index=0

The basic experimental workflow is

$$\text{construct numerical data} \longrightarrow \text{visualize the data}$$

$$\text{observe patterns} \longrightarrow \text{form a conjecture} \longrightarrow \text{verify mathematically.}$$

A graph should therefore be regarded as evidence rather than proof. It may suggest a mathematical property, but the final conclusion should still be supported by analysis whenever possible.

For example, comparing a bounded oscillatory function with the same function plus a linear term immediately shows that similar local oscillations do not necessarily imply periodicity. The course uses this type of comparison to illustrate how visualization can reveal qualitative behavior. :contentReference[oaicite:1]index=1

3.2 Basic Two-Dimensional Plots

For sampled data

$$(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n),$$

the basic MATLAB command is `plot(x,y)`. The vectors x and y must have compatible dimensions. If only `plot(y)` is used, MATLAB takes the horizontal coordinates to be $1, 2, \ldots, n$. :contentReference[oaicite:2]index=2

A simple function plot can be generated by first sampling the independent variable and then evaluating the function.

```
x = linspace(0,2*pi,200);
y = sin(x);

plot(x,y)
grid on
```

Several curves may be placed in the same coordinate system.

```
x = linspace(0,2*pi,200);

y1 = sin(x);
y2 = cos(x);

plot(x,y1,x,y2)
legend('sin(x)','cos(x)')
```

Alternatively, `hold on` preserves the existing figure while additional curves are added. Both approaches appear frequently in mathematical experiments. :contentReference[oaicite:3]index=3

```
plot(x,y1)
hold on
plot(x,y2)
hold off
```

3.3 Plot Appearance and Coordinate Control

The visual form of a curve can be controlled by specifying its color, line style, and marker type. The course uses the standard MATLAB abbreviations, including `r`, `g`, `b`, and `k` for colors, together with line styles such as `-`, `--`, `:`, and `-.`. :contentReference[oaicite:4]index=4

```
plot(x,y,'r--','LineWidth',1.5)

xlabel('x')
ylabel('y')
title('Function Plot')
legend('y = f(x)')
grid on
```

The visible coordinate region can be controlled by `axis`.

```
axis([xmin xmax ymin ymax])
```

For three-dimensional figures,

```
axis([xmin xmax ymin ymax zmin zmax])
```

The axes object can also be modified through `gca` and `set`. The course uses this mechanism to change tick positions, labels, fonts, and other graphical properties. :contentReference[oaicite:5]index=5

```
set(gca,'FontSize',14)
set(gca,'YTick',[-1 -0.5 0 0.5 1])
```

3.4 Multiple Figures and Subplots

The command `figure` creates or activates a figure window, while `clf` clears the current figure.

```
figure(1)
clf
```

The command

```
subplot(m,n,k)
```

divides one figure into an $m \times n$ array of axes and activates the k-th position. :contentReference[oaicite:6]index=6
For example:

```
x = linspace(0,2*pi,100);

for k = 1:4
    subplot(2,2,k)
    plot(x,sin(k*x))
    grid on
end
```

Subplots are useful when the purpose of the experiment is comparison rather than the detailed presentation of a single curve.

3.5 Function Plotting and Coordinate Transformations

When a mathematical function is known analytically, MATLAB can also plot it directly over a specified interval. The course uses both `fplot` and the older `ezplot` syntax for this purpose. :contentReference[oaicite:7]index=7
For current shorthand use, `fplot` is the more convenient form.

```
f = @(x) exp(x) + sin(x);

fplot(f,[-1,1])
grid on
```

Logarithmic coordinates are particularly useful when the scale of the data changes rapidly or when power-law and exponential behavior are being investigated. MATLAB provides three standard forms. :contentReference[oaicite:8]index=8

```
semilogx(x,y)        % Logarithmic x-axis
semilogy(x,y)        % Logarithmic y-axis
loglog(x,y)          % Both axes logarithmic
```

For example, if $y = Cx^{\alpha}$, then

$$\log y = \log C + \alpha \log x.$$

Hence a power law becomes approximately linear on a log–log plot, and its slope indicates the exponent α.

3.6 Scatter Plots and Data Visualization

When the data are discrete observations rather than samples from a known function, connecting neighboring points by line segments may be misleading. A scatter plot displays the observations directly.

```
scatter(x,y)
```

The corresponding three-dimensional command is

```
scatter3(x,y,z)
```

The course emphasizes scatter plots as a convenient way to inspect relationships between variables and to reveal geometric patterns in sampled data. :contentReference[oaicite:9]index=9
Other useful data-oriented graphics include:

```
bar(x,y)             % Bar chart
histogram(x)         % Histogram
stem(x,y)            % Stem plot
pie(x)               % Pie chart
area(x,y)            % Area plot
```

The original course also lists several related special graphics such as `errorbar`, `feather`, `fill`, and three-dimensional variants. :contentReference[oaicite:10]index=10

3.7 Polar and Parametric Curves

Some curves are more naturally represented in polar or parametric form.
For a polar curve

$$r = r(\theta),$$

the angle and radius are sampled separately. The course uses the `polar` command for such plots. :contentReference[oaicite:11]index=11
In modern MATLAB notation, one may use:

```
theta = linspace(0,2*pi,500);
r = 2*(1-cos(theta));

polarplot(theta,r)
```

For a parametric planar curve

$$x = x(t), \qquad y = y(t),$$

the ordinary `plot` command is sufficient.

```
t = linspace(0,2*pi,500);

x = cos(t).^3;
y = sin(t).^3;

plot(x,y)
axis equal
```

A three-dimensional parametric curve

$$x = x(t), \qquad y = y(t), \qquad z = z(t)$$

is plotted using `plot3`. :contentReference[oaicite:12]index=12

```
t = linspace(0,4*pi,500);

x = cos(t);
y = sin(t);
z = t;

plot3(x,y,z)
grid on
```

3.8 Surfaces and Mesh Grids

To visualize a function of two variables,

$$z = f(x, y),$$

MATLAB first constructs a rectangular grid in the xy-plane and then evaluates the function on that grid. This gives the standard sequence

$$\begin{aligned} &\text{sample } x, y \longrightarrow \text{construct a grid} \\ &\text{evaluate } f(x, y) \longrightarrow \text{draw the surface.} \end{aligned}$$

The course explicitly presents these as the basic steps of three-dimensional surface plotting. :contentReference[oaicite:13]index=13

```
x = linspace(-2,2,100);
y = linspace(-2,2,100);

[X,Y] = meshgrid(x,y);
Z = sin(X.^2 + Y.^2);

mesh(X,Y,Z)
```

The main surface commands are:

```
mesh(X,Y,Z)
surf(X,Y,Z)
meshc(X,Y,Z)
meshz(X,Y,Z)
```

The conceptual distinction is that `mesh` emphasizes the grid lines, whereas `surf` displays a filled surface. The course also introduces `meshc` and `meshz` as augmented mesh visualizations. :contentReference[oaicite:14]index=14

3.9 Contour Plots

A contour plot represents the level sets

$$f(x, y) = c.$$

It therefore provides a two-dimensional representation of a function of two variables. The basic commands are:

```
contour(X,Y,Z)
contourf(X,Y,Z)
contour3(X,Y,Z)
colorbar
```

The number of contour levels may be prescribed.

```
contour(X,Y,Z,20)
```

If the contour values themselves are required, the plot can be labeled using `clabel`. The course distinguishes ordinary contours, filled contours, and three-dimensional contour representations. :contentReference[oaicite:15]index=15
Contour plots are particularly useful in later optimization experiments because minima, maxima, valleys, and level-set geometry can often be seen directly.

3.10 Vector Fields and Motion

A vector field associates a vector with each point in space. In two dimensions it may be written as

$$\boldsymbol{F}(x, y) = \begin{pmatrix} u(x, y) \\ v(x, y) \end{pmatrix}.$$

MATLAB represents a sampled two-dimensional vector field using `quiver`, and a three-dimensional field using `quiver3`. :contentReference[oaicite:16]index=16

```
[X,Y] = meshgrid(-2:0.2:2);

```

```
U = -Y;
V = X;

quiver(X,Y,U,V)
axis equal
```

The command `gradient` can also be used to approximate derivatives of sampled scalar fields.

```
[Fx,Fy] = gradient(F);
```

Thus visualization can be used not only for functions and data but also for direction fields, gradients, and dynamical behavior.

3.11 Simple Animation

A mathematical experiment sometimes concerns motion rather than a static object. The course introduces `comet` and `comet3` as simple ways to display trajectories dynamically. :contentReference[oaicite:17]index=17
For example:

```
t = linspace(0,5,1000);

x = 10*t;
y = 20*t - 4.9*t.^2;

comet(x,y)
```

More complicated animations can be constructed by discretizing time and redrawing the changing configuration inside a loop. The cycloid experiment in the course follows precisely this idea: discretize the parameter t, update the moving circle and point, and retain the previously generated trajectory. :contentReference[oaicite:18]index=18

3.12 Chapter Shorthand

The most useful visualization commands from this chapter can be summarized as follows.

```
% Basic 2D plot
plot(x,y)
fplot(f,[a,b])
scatter(x,y)

% Multiple curves
plot(x,y1,x,y2)
hold on
hold off
legend('curve 1','curve 2')

% Figure control
figure(1)
clf
close all
subplot(m,n,k)

```

```
% Labels and axes
xlabel('x')
ylabel('y')
zlabel('z')
title('Title')
grid on
axis([xmin xmax ymin ymax])
axis equal

% Logarithmic coordinates
semilogx(x,y)
semilogy(x,y)
loglog(x,y)

% Polar and spatial curves
polarplot(theta,r)
plot3(x,y,z)
scatter3(x,y,z)

% Grid and surfaces
[X,Y] = meshgrid(x,y);
Z = f(X,Y);

mesh(X,Y,Z)
surf(X,Y,Z)

% Contours
contour(X,Y,Z)
contourf(X,Y,Z)
contour3(X,Y,Z)
colorbar

% Vector fields
quiver(X,Y,U,V)
quiver3(X,Y,Z,U,V,W)

% Simple animation
comet(x,y)
comet3(x,y,z)
```

The essential principle is not to create a graph merely because MATLAB makes plotting easy. The representation should be chosen according to the mathematical question: curves for functional behavior, scatter plots for discrete data, logarithmic coordinates for scale and growth, surfaces and contours for multivariable functions, vector fields for directional information, and animation for time-dependent behavior. In a mathematical experiment, visualization is most useful when it leads to a precise question that can subsequently be analyzed or verified.

Chapter 4

Sequences, Series, and Experimental Exploration

4.1 The Experimental Method

A mathematical experiment is useful when a pattern is difficult to recognize directly from a formula. Computation can generate examples, visualization can reveal structure, and the observed structure can suggest a conjecture.
A common workflow is

$$\text{special cases} \longrightarrow \text{numerical observation}$$
$$\text{pattern} \longrightarrow \text{conjecture} \longrightarrow \text{mathematical verification.}$$

The experiment itself is not a proof. Its role is to identify plausible structure and to guide subsequent mathematical reasoning. In particular, special cases often suggest the correct general statement, but numerical agreement alone cannot establish that statement.

4.2 Linear Periodic Functions

A function f is p-periodic if $f(x+p) = f(x)$. Equivalently, $f(x+p) - f(x) = 0$.
The course extends this idea by allowing the difference to be a constant.

Definition 4.2.1. Let $p > 0$. A function f is called *linear p-periodic* if there exists a constant $M \in \mathbb{R}$ such that

$$f(x+p) = f(x) + M$$

for every x. The constant M is called the p-translation constant.

Ordinary p-periodic functions correspond to the special case $M = 0$.
The essential structural observation is that a linear p-periodic function behaves like a periodic function superimposed on a straight line. Indeed, define

$$k = \frac{M}{p}.$$

Then

$$g(x) = f(x) - kx$$

satisfies $g(x+p) = g(x)$. Hence

$$f(x) = kx + g(x),$$

where g is p-periodic.
A typical experiment compares a periodic function with the same function plus a linear trend.

```
x = linspace(0,8*pi,400);

y1 = sin(x + cos(x + sin(x)));
y2 = 0.2*x + y1;

plot(x,y1,x,y2)
legend('periodic','linear periodic')
grid on
```

4.3 Antiderivatives of Periodic Functions

Let h be a continuous p-periodic function and define

$$S(x) = \int_a^x h(t)\,dt.$$

Numerical plots suggest that $S(x)$ oscillates around a straight line rather than remaining periodic.
Since h is p-periodic,

$$S(x+p) - S(x) = \int_x^{x+p} h(t)\,dt = \int_0^p h(t)\,dt.$$

The right-hand side is independent of x. Therefore S is linear p-periodic.
Its linear trend has slope

$$k = \frac{1}{p}\int_0^p h(t)\,dt,$$

which is exactly the mean value of h over one period.
Hence

$$S(x) - kx$$

is p-periodic.
This illustrates the experimental strategy particularly clearly: first plot the antiderivative, observe an approximately periodic oscillation around a line, estimate the line, and then derive its slope theoretically.

4.4 Fibonacci Sequence and Exponential Growth

The Fibonacci sequence is defined by

$$F_1 = F_2 = 1, \qquad F_{n+2} = F_{n+1} + F_n.$$

The first experiment is simply to generate sufficiently many terms and observe their growth.

```
N = 30;
F = zeros(1,N);

F(1) = 1;
F(2) = 1;

for n = 3:N
    F(n) = F(n-1) + F(n-2);
end

plot(1:N,F)
```

The sequence increases rapidly, suggesting exponential rather than polynomial growth. To test this hypothesis, plot $\log F_n$ against n.

```
n = 1:N;

plot(n,log(F))
grid on
```

The resulting points are approximately linear. If

$$\log F_n \approx an + b,$$

then

$$F_n \approx Ce^{an}.$$

Thus a nearly straight logarithmic plot suggests exponential growth.
To determine the exponential factor theoretically, assume temporarily that $F_n \approx Cr^n$. Substitution into the recurrence gives

$$r^2 = r + 1.$$

Therefore

$$r = \frac{1+\sqrt{5}}{2} \quad \text{or} \quad r = \frac{1-\sqrt{5}}{2}.$$

The dominant root is the golden ratio

$$\phi = \frac{1+\sqrt{5}}{2}.$$

Consequently, the numerical experiment suggests

$$F_n \asymp \phi^n.$$

A more precise analysis gives Binet's formula

$$F_n = \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right].$$

Since the second term tends rapidly to zero in magnitude,

$$F_n \sim \frac{\phi^n}{\sqrt{5}}.$$

This example also shows why experimental conjectures must be checked: recognizing the dominant term is easier than recovering the exact formula.

4.5 Harmonic Series and Growth Rate

Consider the partial sums of the harmonic series,

$$H_n = \sum_{k=1}^{n} \frac{1}{k}.$$

The series diverges, but its divergence is extremely slow. A first experiment plots H_n against n.

```
N = 10000;
n = 1:N;

H = cumsum(1./n);

plot(n,H)
grid on
```

The graph grows slowly and resembles a logarithmic curve. This suggests comparing H_n directly with $\log n$.

```
plot(n,H,n,log(n))
legend('H_n','log(n)')
grid on
```

The two curves become nearly parallel, suggesting that $H_n - \log n$ approaches a constant.

```
D = H - log(n);

plot(n,D)
grid on
```

Indeed,

$$H_n - \log n \longrightarrow \gamma,$$

where γ is the Euler–Mascheroni constant. Hence

$$H_n = \log n + \gamma + o(1),$$

so the harmonic partial sums diverge with logarithmic order.
Another useful experiment compares values at different scales, for example $H_{2n} - H_n$. Since

$$H_n \sim \log n,$$

one expects

$$H_{2n} - H_n \longrightarrow \log 2.$$

More generally,

$$H_{kn} - H_n \longrightarrow \log k$$

for fixed $k > 0$. Such scale comparisons are often useful for detecting asymptotic growth without first knowing the exact asymptotic formula.

4.6 Exploring General Sequences and Series

The same approach can be applied to more general sequences and partial sums. For example, consider

$$S_n(\alpha) = \sum_{k=1}^{n} \frac{1}{k^\alpha}.$$

Instead of beginning with a theorem, one may compute $S_n(\alpha)$ for several values of α, compare their graphs, and investigate whether they appear bounded or unbounded.

```
N = 10000;
n = 1:N;

alpha = 0.5;
S = cumsum(1./n.^alpha);

plot(n,S)
grid on
```

Changing α provides experimental evidence for qualitatively different regimes. The resulting conjecture should then be compared with the known convergence criterion for p-series.
Recurrence relations can be investigated in the same way. A combinatorial counting problem may first generate numerical values, after which one looks for a recurrence, identifies a familiar sequence, and finally proves the relation mathematically.

4.7 Linearization as an Experimental Tool

A recurring idea in numerical exploration is to transform nonlinear growth into an approximately linear relationship.
For exponential growth

$$y_n \approx Ce^{an},$$

taking logarithms gives

$$\log y_n \approx an + \log C.$$

For power-law growth

$$y_n \approx Cn^{\alpha},$$

taking logarithms twice gives

$$\log y_n \approx \alpha \log n + \log C.$$

Thus:

```
plot(n,log(y))          % Test exponential growth
loglog(n,y)             % Test power-law growth
```

If the transformed data are approximately linear, the slope provides an estimate of the corresponding growth exponent. This idea connects visualization, asymptotic analysis, and data fitting.

4.8 Chapter Shorthand

The main computational patterns of this chapter are compact.

```
% Generate a recurrence
a = zeros(1,N);
a(1) = a1;
a(2) = a2;

for n = 3:N
    a(n) = a(n-1) + a(n-2);
end

% Partial sums
n = 1:N;
S = cumsum(1./n);

% Compare growth
plot(n,S)
plot(n,log(S))
semilogy(n,S)
loglog(n,S)
```

```
19
20 % Compare two candidate asymptotic expressions
21 plot(n,S,n,g(n))
22 legend('sequence','comparison')
23
24 % Examine a difference
25 D = S - g(n);
26 plot(n,D)
27
28 % Linear fit to transformed data
29 p = polyfit(n,log(a),1);
30 slope = p(1);
```

The main lesson of these experiments is methodological. Numerical computation is especially useful for discovering qualitative structure: periodicity, growth rates, asymptotic behavior, and recurrence patterns. A useful experiment does not end when a graph looks convincing. It should lead to a precise mathematical statement that can then be tested, refined, and, when possible, proved.

Chapter 5

Matrix Experiments and Linear Algebra

5.1 Matrices as Mathematical Models

Matrices provide a compact representation of systems containing many interacting quantities. In mathematical experiments, a matrix may represent a system of equations, a linear transformation, a transition rule, or a network of relationships.
A large class of problems can be reduced to a linear system

$$A\boldsymbol{x} = \boldsymbol{b}.$$

Once the model has been formulated, MATLAB solves the system directly by

```
x = A\b;
```

The backslash operator should normally be preferred to explicitly computing `inv(A)*b`.
One experiment in the course reconstructs an asteroid orbit from several observed points. The unknown parameters in the equation of the orbit become entries of a vector, while substituting the measured coordinates produces a linear system. Thus a geometric parameter-estimation problem becomes a matrix problem. :contentReference[oaicite:1]index=1
The general modeling pattern is

$$\text{observations or conditions} \longrightarrow \text{linear equations}$$
$$\text{matrix equation} \longrightarrow \text{numerical solution}.$$

5.2 Linear Iteration

Another fundamental matrix experiment is generated by

$$\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k.$$

Starting from an initial vector $\boldsymbol{x}_0$, repeated multiplication gives

$$\boldsymbol{x}_k = A^k\boldsymbol{x}_0.$$

A simple MATLAB implementation is

```
x = x0;

for k = 1:N
    x = A*x;
end
```

In two dimensions, the iterates can be plotted as points or vectors. The course experiment observes that, for suitable matrices and most initial vectors, the iterates gradually approach a particular line through the origin. :contentReference[oaicite:2]index=2
This numerical phenomenon suggests that repeated multiplication by A selects a preferred direction. That direction is determined by an eigenvector.

5.3 Eigenvalues and Dominant Directions

Suppose A has linearly independent eigenvectors $\boldsymbol{v}_1, \dots, \boldsymbol{v}_n$, with

$$A\boldsymbol{v}_i = \lambda_i\boldsymbol{v}_i.$$

Write the initial vector as

$$\boldsymbol{x}_0 = c_1\boldsymbol{v}_1 + \cdots + c_n\boldsymbol{v}_n.$$

Then

$$A^k \boldsymbol{x}_0 = c_1 \lambda_1^k \boldsymbol{v}_1 + \cdots + c_n \lambda_n^k \boldsymbol{v}_n.$$

If

$$|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n|$$

and $c_1 \neq 0$, then for large k,

$$A^k \boldsymbol{x}_0 \approx c_1 \lambda_1^k \boldsymbol{v}_1.$$

Hence the iterates become approximately parallel to $\boldsymbol{v}_1$.
This explains the experimental observation that the points generated by $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k$ approach a fixed direction. The course explicitly uses `eig` to compare the observed limiting line with the eigenvectors of the matrix. :contentReference[oaicite:3]index=3

```
[V,D] = eig(A);
```

The columns of `V` are eigenvectors, and the diagonal entries of `D` are the corresponding eigenvalues.

5.4 Diagonalization

If A has n linearly independent eigenvectors, let

$$P = \begin{bmatrix} \boldsymbol{v}_1 & \boldsymbol{v}_2 & \cdots & \boldsymbol{v}_n \end{bmatrix}, \qquad D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n).$$

Then

$$A = PDP^{-1}.$$

Consequently,

$$A^k = PD^kP^{-1}.$$

Since powers of a diagonal matrix are easy to compute,

$$D^k = \operatorname{diag}(\lambda_1^k, \ldots, \lambda_n^k),$$

diagonalization converts repeated matrix multiplication into scalar powers.
The course emphasizes this connection between diagonalization, eigenvalues, eigenvectors, and the long-term behavior of matrix iterations. :contentReference[oaicite:4]index=4
In MATLAB,

```
[P,D] = eig(A);

Aapprox = P*D/P;
```

If there are fewer than n linearly independent eigenvectors, ordinary diagonalization is impossible. The course mentions the Jordan form only as a possible further reduction and does not develop its theory. :contentReference[oaicite:5]index=5

5.5 The Power Method

The power method converts the preceding observation into an algorithm. Its purpose is to approximate the eigenvalue of largest modulus and a corresponding eigenvector. :contentReference[oaicite:6]index=6
Starting from a nonzero vector $\boldsymbol{x}_0$, repeatedly compute

$$\boldsymbol{y}_{k+1} = A\boldsymbol{x}_k.$$

Direct iteration may produce vectors whose entries become extremely large or small. Therefore each iterate is normalized. The course specifically introduces normalization to avoid overflow and underflow. :contentReference[oaicite:7]index=7
A convenient version is

$$\boldsymbol{y}_{k+1} = A\boldsymbol{x}_k, \qquad \mu_{k+1} = \|\boldsymbol{y}_{k+1}\|_\infty ,$$

followed by

$$\boldsymbol{x}_{k+1} = \frac{\boldsymbol{y}_{k+1}}{\mu_{k+1}}.$$

A compact MATLAB implementation is

```
x = x0;
x = x/norm(x,inf);

for k = 1:maxIter
    y = A*x;
    mu = norm(y,inf);
    xnew = y/mu;

    if norm(xnew-x,inf) < tol
        break
    end

    x = xnew;
end
```

After convergence, $\boldsymbol{x}_k$ approximates a dominant eigenvector. For a more reliable estimate of the eigenvalue, one may use the Rayleigh quotient

$$\lambda \approx \frac{\boldsymbol{x}^T A\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}}.$$

```
lambda = (x'*A*x)/(x'*x);
```

5.6 Convergence of the Power Method

Assume

$$|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n| ,$$

and the initial vector has a nonzero component in the direction $\boldsymbol{v}_1$. Then

$$A^k\boldsymbol{x}_0 = \lambda_1^k \left[c_1\boldsymbol{v}_1 + \sum_{j=2}^{n} c_j \left(\frac{\lambda_j}{\lambda_1}\right)^k \boldsymbol{v}_j \right].$$

Since

$$\left|\frac{\lambda_j}{\lambda_1}\right| < 1,$$

the remaining components decay relative to the dominant component. Thus the direction of the normalized iterates approaches $\boldsymbol{v}_1$.
The convergence rate is governed mainly by

$$\left|\frac{\lambda_2}{\lambda_1}\right|.$$

A small ratio gives fast separation of the dominant direction, while a ratio close to 1 leads to slow convergence.

5.7 Inverse Power Method

The power method finds an eigenvalue of largest modulus. To locate an eigenvalue of smallest modulus, one can instead apply the power method to A^{-1}.
If

$$A\boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i,$$

then

$$A^{-1}\boldsymbol{v}_i = \frac{1}{\lambda_i}\boldsymbol{v}_i.$$

Therefore the eigenvalue of A having smallest modulus corresponds to the eigenvalue of A^{-1} having largest modulus.
The course emphasizes that A^{-1} should not be formed explicitly. Instead, solve a linear system at each iteration. :contentReference[oaicite:8]index=8

```
x = x0;
x = x/norm(x,inf);

for k = 1:maxIter
    y = A\x;
    xnew = y/norm(y,inf);

    if norm(xnew-x,inf) < tol
        break
    end

    x = xnew;
end
```

The corresponding eigenvalue of A can again be estimated using the Rayleigh quotient.

5.8 Transition Matrices and Long-Term Behavior

Matrix iteration also appears naturally in systems whose state changes from one stage to the next. If $\boldsymbol{x}_k$ represents the state at step k, then a model may take the form

$$\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k.$$

The course uses a market-share problem as an example of this type of matrix model. :contentReference[oaicite:9]index=9
After k steps,

$$\boldsymbol{x}_k = A^k\boldsymbol{x}_0.$$

Thus long-term behavior is again governed by the eigenvalues and eigenvectors of A.
If the iteration converges to a steady state $\boldsymbol{x}^*$, then

$$A\boldsymbol{x}^* = \boldsymbol{x}^*,$$

so the equilibrium is an eigenvector corresponding to eigenvalue 1.
The same algebraic structure therefore connects matrix models, dynamical iteration, and eigenvalue computation.

5.9 Matrices and Graphs

A matrix can also encode relationships between objects. For a graph with vertices $1, \ldots, n$, its adjacency matrix is defined by

$$A_{ij} = \begin{cases} 1, & \text{if vertex } i \text{ is connected to vertex } j, \\ 0, & \text{otherwise.} \end{cases}$$

The course uses a social-relationship example in which each row of A records one resident's choices of friends and asks for the graph represented by this adjacency matrix. :contentReference[oaicite:10]index=10

A useful matrix interpretation is that

$$(A^2)_{ij}$$

counts walks of length 2 from vertex i to vertex j. More generally,

$$(A^k)_{ij}$$

counts walks of length k.

In MATLAB, the corresponding graph can be displayed by

```
G = graph(A);
plot(G)
```

For a directed adjacency matrix, use

```
G = digraph(A);
plot(G)
```

5.10 Permutation Parity and the Fifteen Puzzle

The course also uses computation to explore the inversion number of a permutation. For a permutation

$$\pi = (a_1, a_2, \ldots, a_n),$$

an inversion is a pair $i < j$ for which $a_i > a_j$. The inversion number is

$$I(\pi) = \#\{(i, j) : i < j,\ a_i > a_j\}.$$

A permutation is even when $I(\pi)$ is even and odd when $I(\pi)$ is odd. The course asks students to implement the inversion count and experimentally observe how a transposition changes permutation parity. :contentReference[oaicite:11]index=11

A direct implementation is

```
invNum = 0;

for i = 1:n-1
    for j = i+1:n
        if p(i) > p(j)
            invNum = invNum + 1;
        end
    end
end
```

This idea is then connected with the fifteen-puzzle: the movement of tiles is translated into changes in a permutation, and parity gives information about which configurations are reachable. The course explicitly asks students to observe the effect of legal movements on permutation parity. :contentReference[oaicite:12]index=12

5.11 Chapter Shorthand

The principal matrix commands and experimental patterns are:

```
% Linear system
x = A\b;

% Basic matrix information
det(A)
rank(A)
trace(A)
rref(A)

% Eigenvalues and eigenvectors
lambda = eig(A);
[V,D] = eig(A);

% Singular value decomposition
[U,S,V] = svd(A);

% Matrix iteration
x = x0;

for k = 1:N
    x = A*x;
end

% Power method
x = x0/norm(x0,inf);

for k = 1:maxIter
    y = A*x;
    xnew = y/norm(y,inf);

    if norm(xnew-x,inf) < tol
        break
    end

    x = xnew;
end

lambda = (x'*A*x)/(x'*x);

% Inverse power iteration
y = A\x;

% Matrix powers
Ak = A^k;

% Graph from adjacency matrix
G = graph(A);
plot(G)

```

```
% Directed graph
G = digraph(A);
plot(G)
```

The central idea of this chapter is that a matrix is not merely a rectangular array of numbers. It may encode a mathematical model, and repeated matrix multiplication reveals its intrinsic structure. Eigenvalues determine rates of expansion or contraction, eigenvectors determine preferred directions, and the power method turns this qualitative observation into a practical numerical algorithm.

Chapter 6

Stochastic Simulation and Monte Carlo Methods

6.1 Random Simulation

Many mathematical problems involve uncertainty or randomness. When an exact probabilistic calculation is difficult, one may simulate the random mechanism many times and estimate the quantity of interest from the generated samples.
The basic Monte Carlo principle is

$$\text{random model} \longrightarrow \text{independent simulation}$$
$$\text{sample statistic} \longrightarrow \text{approximation of the target quantity.}$$

For an event A, suppose N independent experiments are performed and the event occurs N_A times. Then

$$\widehat{P}(A) = \frac{N_A}{N}$$

is the empirical estimate of $P(A)$.
Similarly, if $X_1, \ldots, X_N$ are simulated copies of a random variable X, then

$$\widehat{\mathbb{E}[X]} = \frac{1}{N} \sum_{i=1}^{N} X_i$$

approximates the expectation of X.

6.2 Basic Random Number Generation

MATLAB provides several fundamental random-number generators. The course emphasizes four basic commands: `rand`, `randn`, `randi`, and `randperm`. :contentReference[oaicite:1]index=1

```
rand(m,n)            % Uniform distribution on (0,1)
randn(m,n)           % Standard normal distribution
randi(N,m,n)         % Integers from 1 to N
randperm(N)          % Random permutation of 1,...,N
```

For example:

```
u = rand(1,1000);
z = randn(1,1000);

mean(u)
var(u)

mean(z)
var(z)
```

For sufficiently large samples, the results should be close to the theoretical values

$$\mathbb{E}[U] = \frac{1}{2}, \qquad \mathrm{Var}(U) = \frac{1}{12}$$

for $U \sim U(0,1)$, and

$$\mathbb{E}[Z] = 0, \qquad \mathrm{Var}(Z) = 1$$

for $Z \sim N(0,1)$.

6.3 Random Seeds and Reproducibility

Computer-generated random numbers are actually pseudorandom. Their sequence depends on the internal random seed.
A fixed seed makes an experiment reproducible.

```
rng(1)

x = rand(1,5);
```

Repeating the same commands with the same seed produces the same random sequence. MATLAB also provides

```
rng('default')
rng('shuffle')
```

The first restores the default random-number state, while the second initializes the generator from the current system state. The course explicitly emphasizes that fixing the seed is useful when a simulation must be reproduced. :contentReference[oaicite:2]index=2

6.4 Simulating Common Distributions

Besides uniform and standard normal variables, MATLAB can directly generate many common probability distributions.
The course uses the general form

```
R = random(dist,p1,p2,...,m,n);
```

where `dist` specifies the distribution and the following arguments are its parameters. Distributions listed in the course include binomial, uniform, normal, Poisson, chi-square, t, F, geometric, hypergeometric, exponential, gamma, and Weibull distributions. :contentReference[oaicite:3]index=3
Frequently used specialized commands include:

```
binornd(n,p,m,k)
unifrnd(a,b,m,k)
normrnd(mu,sigma,m,k)
poissrnd(lambda,m,k)
exprnd(mu,m,k)
```

For multivariate normal data:

```
R = mvnrnd(mu,Sigma,N);
```

Each row of `R` is one simulated observation. The course uses this to illustrate correlated multidimensional normal variables. :contentReference[oaicite:4]index=4

6.5 Frequency Approximation of Probability

Suppose I_i is the indicator of success in the i-th experiment:

$$I_i = \begin{cases} 1, & \text{success}, \\ 0, & \text{failure}. \end{cases}$$

Then

$$\widehat{P}(A) = \frac{1}{N} \sum_{i=1}^{N} I_i.$$

This gives the standard MATLAB pattern

```
success = 0;

for k = 1:N
    if experiment_is_successful
        success = success + 1;
    end
end

p = success/N;
```

Whenever possible, vectorization is more efficient.

```
success = condition_on_all_samples;
p = mean(success);
```

Because logical values are represented numerically by 0 and 1, `mean(success)` directly computes the empirical success probability.

6.6 Geometric Probability: The Meeting Problem

Suppose two people independently arrive during the time interval $[0, T]$. The first person waits for time t, where $0 \le t \le T$.
Let X and Y denote their arrival times. They meet exactly when

$$|X - Y| \le t.$$

Thus the sample space is the square

$$0 \le X, Y \le T,$$

and the event corresponds to the band

$$|X - Y| \le t.$$

The course uses this problem to connect Monte Carlo simulation with geometric probability. :contentReference[oaicite:5]index=5
A vectorized simulation is:

```
N = 1e5;

x = T*rand(N,1);
y = T*rand(N,1);

success = abs(x-y) <= t;

p = mean(success);
```

The theoretical probability is

$$P(A) = 1 - \left(1 - \frac{t}{T}\right)^2.$$

The experiment should approach this value as N becomes large.

6.7 Simulation of Sequential Random Processes

Some stochastic experiments do not consist of one random draw. Instead, a random process continues until a stopping condition is reached.
The Banach matchbox problem is a representative example. Two boxes initially contain n matches each. At every step, one box is selected randomly and one match is removed. The experiment stops when an empty box is selected. :contentReference[oaicite:6]index=6
The computational structure is therefore:

```
while true
    choice = randi(2);

    if choice == 1
        box1 = box1 - 1;
    else
        box2 = box2 - 1;
    end

    if box1 < 0 || box2 < 0
        break
    end
end
```

An outer loop repeats this random process many times and estimates the desired probability from the frequency of the target terminal state.
This illustrates an important distinction:

$$\text{Monte Carlo experiment} = \text{many independent sample paths.}$$

Each sample path may itself contain a random number of steps.

6.8 Sampling Without Replacement

The course also considers a drawing-lots problem with n people and a winning tickets. Each person draws once without replacement.
A simple simulation may choose the winning positions using

```
winner = randperm(n,a);
```

Then the event that person r receives a prize is

```
success = any(winner == r);
```

Repeating this experiment estimates the probability. The theoretical result is

$$P(\text{person } r \text{ wins}) = \frac{a}{n},$$

which is independent of r. Therefore the procedure is fair. :contentReference[oaicite:7]index=7
This example shows that Monte Carlo simulation is useful not only for obtaining a numerical answer, but also for checking symmetry and fairness.

6.9 Monte Carlo Estimation of Expectations

For a random variable $Y = g(X)$,

$$\mathbb{E}[Y] = \mathbb{E}[g(X)]$$

can be estimated by

$$\frac{1}{N}\sum_{i=1}^{N} g(X_i).$$

The MATLAB pattern is simply

```
X = simulate_samples(N);
Y = g(X);

estimate = mean(Y);
```

The course uses a miner escape problem as an example of this principle. At each visit to the starting point, the miner randomly chooses one of three tunnels. One tunnel leads to safety, while the others return him to the starting point after different amounts of time. The experiment records the total time until escape and averages this quantity over many repeated trials. :contentReference[oaicite:8]index=8
Thus a random stopping-time problem can be estimated by simulation even before its expectation is derived analytically.

6.10 Simulation-Based Decision Making

Monte Carlo methods can also evaluate decisions under uncertainty.
The course considers a newspaper vendor who chooses a daily purchase quantity q, while demand D is random and approximately normally distributed. For each candidate q, many demand samples are generated and the average profit is estimated. The value of q with the highest simulated mean profit is selected. :contentReference[oaicite:9]index=9
The experimental structure is

$$\text{choose decision } q \longrightarrow \text{simulate random demand}$$
$$\text{compute profit} \longrightarrow \text{estimate mean profit} \longrightarrow \text{compare decisions.}$$

A generic MATLAB implementation is:

```
demand = normrnd(mu,sigma,N,1);

for j = 1:length(qList)
    q = qList(j);

    sold = min(q,demand);
    left = max(q-demand,0);

    profit = price*sold ...
        + salvage*left - cost*q;

    meanProfit(j) = mean(profit);
end

[bestProfit,idx] = max(meanProfit);
bestQ = qList(idx);
```

This idea reappears later in optimization: simulation evaluates an objective function when randomness prevents a simple deterministic formula.

6.11 Monte Carlo for Testing Conjectures

Random simulation may also be used as a rapid test of a mathematical conjecture. The course explicitly presents Monte Carlo sampling as a way to explore whether a proposed statement appears plausible before searching for a rigorous proof. :contentReference[oaicite:10]index=10

For a conjecture involving variables in a region D, generate many random points

$$\boldsymbol{X}_1, \ldots, \boldsymbol{X}_N \in D$$

and test the desired condition at each point.

```
X = random_points(N);

value = test_expression(X);

min(value)
max(value)
```

If a counterexample is found, the conjecture is false. If no counterexample is found, the experiment provides evidence only; it does not prove the conjecture.

6.12 Accuracy and Sample Size

Monte Carlo estimates fluctuate because they are based on random samples.

For Bernoulli trials with success probability p,

$$\widehat{p} = \frac{1}{N} \sum_{i=1}^{N} I_i,$$

and

$$\operatorname{Var}(\widehat{p}) = \frac{p(1-p)}{N}.$$

Thus the typical error decreases on the scale

$$O(N^{-1/2}).$$

Consequently, reducing the random error by a factor of 10 usually requires about 100 times as many simulations.

A useful numerical experiment is to examine how an estimate changes as N increases.

```
Nlist = [100 1000 10000 100000];

for j = 1:length(Nlist)
    N = Nlist(j);

    X = simulate(N);
    estimate(j) = mean(X);
end
```

The important quantities in a simulation are therefore not only the estimate, but also the sample size and the variability of the estimate.

6.13 Chapter Shorthand

The principal commands and patterns are:

```
% Random seed
rng(1)
rng('default')
rng('shuffle')

% Basic random numbers
rand(m,n)
randn(m,n)
randi(N,m,n)
randperm(N)

% Common distributions
unifrnd(a,b,m,n)
normrnd(mu,sigma,m,n)
binornd(k,p,m,n)
poissrnd(lambda,m,n)
exprnd(mu,m,n)

% Multivariate normal
R = mvnrnd(mu,Sigma,N);

% General distribution syntax
R = random(dist,p1,p2,m,n);

% Empirical probability
success = condition;
p = mean(success);

% Empirical expectation
estimate = mean(X);

% Sample statistics
mean(X)
var(X)
std(X)
corrcoef(X,Y)

% Sampling without replacement
winner = randperm(n,a);

% Basic Monte Carlo structure
result = zeros(N,1);

for k = 1:N
    result(k) = one_simulation();
end

estimate = mean(result);
```

The essential idea of stochastic simulation is to replace an analytically difficult probabilistic calculation by repeated random experiments. The simulation must reproduce the probabilistic mechanism correctly, the number of trials must be sufficiently large, and the resulting numerical evidence should be compared with theoretical results whenever such results are available.

Chapter 7

Computing π and Convergence Experiments

7.1 One Quantity, Different Algorithms

The constant π provides a convenient setting for comparing numerical methods. The objective is not merely to obtain a decimal approximation, but to examine how different algorithms approach the same exact value.
The basic experimental questions are:

- Does the approximation converge to the correct value?
- How fast does the error decrease?
- How much computation is required for a given accuracy?
- Does the method remain useful for more complicated problems?

Thus the general comparison is

$$\text{mathematical identity} \longrightarrow \text{approximation method}$$
$$\text{numerical result} \longrightarrow \text{error and efficiency comparison.}$$

7.2 Taylor Series for the Arctangent

For $|x| \leq 1$, the arctangent has the expansion

$$\arctan x = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \cdots .$$

Since $\arctan 1 = \pi/4$, one immediately obtains

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots .$$

Therefore,

$$\pi \approx 4 \sum_{k=0}^{n} \frac{(-1)^k}{2k+1}.$$

A direct MATLAB implementation is:

```
n = 20000;
s = 0;

for k = 0:n
    s = s + (-1)^k/(2*k+1);
end

piApprox = 4*s;
err = abs(piApprox-pi);
```

Although the formula is mathematically correct, the convergence is slow. The course uses this experiment to emphasize that correctness alone does not make an algorithm computationally efficient.

7.3 Why the Direct Series Converges Slowly

For an alternating series, the truncation error is controlled by the first omitted term. For the approximation

$$S_n(x) = \sum_{k=0}^{n} \frac{(-1)^k x^{2k+1}}{2k+1},$$

the error satisfies approximately

$$|\arctan x - S_n(x)| \leq \frac{|x|^{2n+3}}{2n+3}.$$

At $x = 1$, this bound decreases only like $1/n$. Hence a very large number of terms is required to obtain high precision.
If $|x| \ll 1$, however, the factor $|x|^{2n+3}$ decays rapidly. This suggests a basic acceleration principle:

rewrite the target using rapidly convergent quantities

The course therefore replaces $\arctan 1$ by arctangent values with much smaller arguments.

7.4 A Faster Arctangent Decomposition

Let

$$\alpha = \arctan \frac{1}{2}.$$

Using the tangent subtraction formula gives

$$\frac{\pi}{4} = \arctan \frac{1}{2} + \arctan \frac{1}{3}.$$

Hence

$$\pi = 4\left(\arctan \frac{1}{2} + \arctan \frac{1}{3}\right).$$

Both Taylor expansions now involve numbers strictly smaller than 1, so their terms decrease much more rapidly.
A reusable arctangent approximation is:

```
function s = atanTaylor(x,n)
    s = 0;

    for k = 0:n
        term = (-1)^k*x^(2*k+1)/(2*k+1);
        s = s + term;
    end
end
```

Then

```
n = 30;

a = atanTaylor(1/2,n);
b = atanTaylor(1/3,n);

piApprox = 4*(a+b);
err = abs(piApprox-pi);
```

7.5 Machin's Formula

A still faster identity used in the course is Machin's formula

$$\pi = 16 \arctan \frac{1}{5} - 4 \arctan \frac{1}{239}.$$

The arguments 1/5 and 1/239 are small, so their Taylor expansions converge rapidly. This gives a substantial improvement over the direct Gregory series.
Using the previous function:

```
n = 20;

a = atanTaylor(1/5,n);
b = atanTaylor(1/239,n);

piApprox = 16*a - 4*b;
err = abs(piApprox-pi);
```

The important idea is more general than this particular formula. If two equivalent representations of a quantity are available, their numerical behavior may be very different. Algebraic reformulation can therefore be an essential part of algorithm design.

7.6 Error and Convergence Experiments

To compare two methods, define the absolute error by

$$E_n = |\pi_n - \pi|.$$

One can calculate the approximation for increasing values of n and record the error.

```
N = 30;
err1 = zeros(1,N);
err2 = zeros(1,N);

for n = 1:N
    s1 = atanTaylor(1,n);
    p1 = 4*s1;

    a = atanTaylor(1/5,n);
    b = atanTaylor(1/239,n);
    p2 = 16*a - 4*b;

    err1(n) = abs(p1-pi);
    err2(n) = abs(p2-pi);
end
```

Because errors can differ by many orders of magnitude, a logarithmic vertical axis is often more informative.

```
semilogy(1:N,err1,1:N,err2)
xlabel('Number of terms')
ylabel('Absolute error')
legend('Direct series','Machin formula')
grid on
```

A steep decrease in the error curve indicates faster convergence.

7.7 Monte Carlo Estimation of π

A completely different approach is based on geometric probability.
Generate points uniformly in the unit square

$$Q = [0,1] \times [0,1].$$

The quarter unit disk inside Q is

$$D = \{(x,y) : x^2 + y^2 \leq 1\}.$$

Since

$$\frac{\text{Area}(D)}{\text{Area}(Q)} = \frac{\pi}{4},$$

if N_D of N random points fall inside the quarter disk, then

$$\frac{N_D}{N} \approx \frac{\pi}{4}.$$

Thus

$$\pi \approx 4\frac{N_D}{N}.$$

The MATLAB implementation is particularly simple.

```
N = 100000;

x = rand(N,1);
y = rand(N,1);

inside = x.^2 + y.^2 <= 1;

piApprox = 4*mean(inside);
err = abs(piApprox-pi);
```

The experiment can also be visualized.

```
scatter(x(inside),y(inside),5,'filled')
axis equal
xlim([0 1])
ylim([0 1])
```

7.8 Accuracy of the Monte Carlo Method

The random-point estimator can be written as

$$\widehat{\pi}_N = \frac{4}{N}\sum_{i=1}^{N} I_i,$$

where

$$I_i = \begin{cases} 1, & X_i^2 + Y_i^2 \leq 1, \\ 0, & \text{otherwise.} \end{cases}$$

Since the I_i are Bernoulli random variables, the random error decreases on the scale

$$O(N^{-1/2}).$$

This is much slower than a rapidly convergent deterministic series. Increasing N by a factor of 100 typically improves the Monte Carlo error by only about a factor of 10.
A simple experiment is:

```
Nlist = [1000 10000 100000 1000000];

for j = 1:length(Nlist)
    N = Nlist(j);

    x = rand(N,1);
    y = rand(N,1);

    inside = x.^2 + y.^2 <= 1;
    estimate(j) = 4*mean(inside);
end

error = abs(estimate-pi);
```

Thus Monte Carlo is not an efficient method if the only objective is to compute π accurately.

7.9 Why Monte Carlo Is Still Useful

The strength of Monte Carlo methods is not high precision for simple one-dimensional problems. Their advantage appears when a region is easy to test for membership but difficult to describe analytically.
Suppose a complicated region G lies inside a rectangle Q. If random points are sampled uniformly from Q, then

$$\frac{N_G}{N} \approx \frac{\text{Area}(G)}{\text{Area}(Q)}.$$

Hence

$$\text{Area}(G) \approx \text{Area}(Q)\frac{N_G}{N}.$$

The boundary of G does not need to be explicitly parameterized. One only needs a logical test for whether each point lies in G.
For example, if G is the common region of several disks:

```
inside = true(N,1);

for j = 1:m
    test = (x-a(j)).^2 + (y-b(j)).^2 <= r(j)^2;
    inside = inside & test;
end

areaApprox = areaQ*mean(inside);
```

This is the main reason the course retains Monte Carlo methods despite their relatively slow convergence.

7.10 Buffon's Needle Experiment

A second probabilistic method for estimating π is Buffon's needle experiment.
Consider parallel lines separated by distance d, and a needle of length $\ell < d$. Let X be the distance from the needle center to the nearest parallel line, and let Θ be the angle between the needle and the parallel lines.

The needle crosses a line when

$$X \le \frac{\ell}{2} \sin \Theta.$$

For uniformly random position and orientation, the theoretical crossing probability is

$$P(\text{crossing}) = \frac{2\ell}{\pi d}.$$

If M crossings occur in N simulated throws, then

$$\frac{M}{N} \approx \frac{2\ell}{\pi d},$$

so

$$\pi \approx \frac{2\ell N}{dM}.$$

A simulation can be written as:

```
N = 100000;

X = (d/2)*rand(N,1);
theta = (pi/2)*rand(N,1);

cross = X <= (ell/2)*sin(theta);

p = mean(cross);
piApprox = 2*ell/(d*p);
```

This experiment illustrates a striking principle: a geometric constant may be recovered from the frequency of a random physical event.

7.11 Deterministic and Stochastic Methods

The experiments in this chapter illustrate two fundamentally different approaches.
A deterministic method generates

$$\pi_1, \pi_2, \ldots$$

from a fixed formula. Repeating the computation with the same parameters produces the same result.
A Monte Carlo method produces a random estimate

$$\widehat{\pi}_N,$$

so repeated runs produce slightly different values.
For this problem:

Taylor or Machin method	fast and deterministic,
Monte Carlo method	slow but geometrically flexible.

The appropriate method therefore depends not only on accuracy, but also on the structure of the problem being solved.

7.12 Chapter Shorthand

The main computational patterns are:

```
% Direct Gregory series
s = 0;

for k = 0:n
    s = s + (-1)^k/(2*k+1);
end

piApprox = 4*s;

% Taylor approximation of arctan(x)
s = 0;

for k = 0:n
    s = s + (-1)^k*x^(2*k+1)/(2*k+1);
end

% Machin formula
a = atanTaylor(1/5,n);
b = atanTaylor(1/239,n);

piApprox = 16*a - 4*b;

% Absolute error
err = abs(piApprox-pi);

% Timing
tic
% computation
toc

% Error curve
semilogy(nList,error)

% Random-point Monte Carlo
x = rand(N,1);
y = rand(N,1);

inside = x.^2 + y.^2 <= 1;
piApprox = 4*mean(inside);

% Buffon's needle
X = (d/2)*rand(N,1);
theta = (pi/2)*rand(N,1);

cross = X <= (ell/2)*sin(theta);
piApprox = 2*ell/(d*mean(cross));
```

The central lesson is that numerical computation is not determined by a formula alone. Equivalent mathematical formulas may have dramatically different convergence rates, and

a method that is inefficient for one simple problem may become valuable when applied to a more complicated geometry. Accuracy, convergence, computational cost, and generality should therefore be considered together when evaluating an algorithm.

Chapter 8

Symbolic Computation

8.1 Numerical and Symbolic Computation

Ordinary MATLAB computation is primarily numerical. A quantity such as π^2 is normally represented by a floating-point approximation.
Symbolic computation instead manipulates mathematical expressions themselves. Variables, functions, matrices, derivatives, integrals, and equations can be stored as symbolic objects and transformed algebraically.
The Symbolic Math Toolbox therefore supports two complementary tasks:

$$\text{exact symbolic manipulation} \longrightarrow \text{analytic expression},$$
$$\text{arbitrary-precision evaluation} \longrightarrow \text{high-precision number}.$$

Symbolic computation is particularly useful when an exact expression is more important than a floating-point approximation.

8.2 Symbolic Variables and Expressions

The command `syms` defines symbolic variables.

```
syms x y z

f = x^2 + 2*x*y + y^2;
```

Several variables can be declared simultaneously. Assumptions may also be included.

```
syms x y real
syms n positive
```

The command `sym` converts a number or expression into a symbolic object.

```
a = sym(2);
b = sym(pi);
```

The distinction between numerical and symbolic values is important. For example:

```
a = 2^10000;
b = sym(2)^10000;
```

The numerical expression may overflow, whereas the symbolic expression can retain the exact integer. The course uses this example to illustrate one of the main advantages of symbolic computation. :contentReference[oaicite:1]index=1

8.3 Substitution

The command `subs` replaces symbolic variables by other symbolic or numerical quantities. If

$$f(x, y) = x^2 + xy + y^2,$$

then substituting $x = 2$ gives a symbolic expression in y.

```
syms x y

f = x^2 + x*y + y^2;

g = subs(f,x,2);
```

Several variables can be substituted simultaneously.

```
value = subs(f,[x y],[2 3]);
```

This is useful when an analytic formula is first derived symbolically and then evaluated for particular parameter values.

8.4 Simplification and Algebraic Manipulation

A symbolic expression may admit several algebraically equivalent forms. MATLAB provides commands for common transformations.

```
simplify(expr)
factor(expr)
expand(expr)
collect(expr,x)
```

For example:

```
syms x

f = x^3 + 3*x^2 + 3*x + 1;

factor(f)
expand((x+1)^3)
```

The course also introduces `numden`, which extracts the numerator and denominator of a symbolic rational expression. :contentReference[oaicite:2]index=2

```
[n,d] = numden(expr);
```

Symbolic simplification is especially useful after differentiation, integration, matrix operations, or equation solving, where MATLAB may return an expression that is mathematically correct but unnecessarily complicated.

8.5 Symbolic Functions

A symbolic function explicitly records its independent variables.

```
syms f(x)

f(x) = exp(-x^2);
```

Functions of several variables are defined similarly.

```
syms f(x,y)

f(x,y) = x^2 + y^2;
```

The course also introduces inverse and composite functions.

```
g = finverse(f);
h = compose(f,g);
```

For short symbolic work, direct substitution often provides the same effect as explicit function composition.

8.6 Exact and Arbitrary-Precision Values

The command `vpa` performs variable-precision arithmetic.

```
syms x

a = vpa(pi,20);
b = vpa(pi,50);
```

The precision can also be controlled globally by

```
digits(50)
```

A symbolic object can be converted back to an ordinary double-precision number by

```
x = double(expr);
```

The course contrasts ordinary floating-point arithmetic with symbolic high-precision arithmetic and emphasizes that `vpa` can evaluate quantities beyond ordinary machine precision. :contentReference[oaicite:3]index=3
Thus the typical pattern is

$$\text{exact expression} \longrightarrow \texttt{vpa} \longrightarrow \text{high-precision approximation.}$$

8.7 Limits, Sums, and Products

Symbolic limits are computed using `limit`.

```
syms x

L = limit(sin(x)/x,x,0);
```

One-sided limits can also be specified.

```
L1 = limit(f,x,a,'left');
L2 = limit(f,x,a,'right');
```

Infinite limits are written using `inf`.

```
L = limit(exp(-x),x,inf);
```

Finite or infinite symbolic sums are evaluated using `symsum`.

```
syms k

S = symsum(1/k^2,k,1,inf);
```

Products are handled similarly.

```
P = symprod(k,k,1,n);
```

The course explicitly uses `limit`, `symsum`, and `symprod` as the main commands for symbolic limits, series, and products. :contentReference[oaicite:4]index=4

8.8 Symbolic Differentiation

For a symbolic function f, the first derivative is obtained by

```
df = diff(f,x);
```

The n-th derivative is

```
dnf = diff(f,x,n);
```

For multivariable functions, partial derivatives are computed in the same way.

```
syms x y

f = x^2*exp(-y);

fx = diff(f,x);
fxy = diff(diff(f,x),y);
```

For a vector-valued function

$$\boldsymbol{f} = (f_1, \ldots, f_m)^T,$$

the Jacobian matrix is

$$J_{ij} = \frac{\partial f_i}{\partial x_j}.$$

MATLAB computes it by

```
J = jacobian(f,[x y]);
```

For a scalar function f, the Hessian matrix contains second derivatives:

```
H = hessian(f,[x y]);
```

The course explicitly distinguishes symbolic `diff` from numerical difference operations and includes both Jacobian and Hessian calculations. :contentReference[oaicite:5]index=5

8.9 Taylor Expansion

For a sufficiently smooth function, the Taylor polynomial around $x = a$ is

$$T_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(a)}{k!}(x-a)^k.$$

MATLAB performs the symbolic expansion using `taylor`.

```
syms x

f = exp(x);

T = taylor(f,x,0,'Order',6);
```

An important MATLAB convention is that
“‘latex `'Order',6`

Chapter 9

Nonlinear Equations and Iterative Methods

9.1 Numerical Root Finding

Consider the nonlinear equation

$$f(x) = 0.$$

Unlike linear equations, nonlinear equations generally have no universal analytic solution method. Even when an exact symbolic solution exists, a numerical approximation is often more useful in computation.
The basic numerical strategy is

$$\begin{aligned} &\text{locate or estimate a root} \longrightarrow \text{construct an iteration} \\ &\text{generate approximations} \longrightarrow \text{stop when sufficiently accurate.} \end{aligned}$$

A numerical root-finding method therefore requires an initial value or interval, an update rule, and a stopping criterion.

9.2 Bisection Method

Suppose f is continuous on $[a, b]$ and

$$f(a)f(b) < 0.$$

Then the intermediate value theorem guarantees at least one root in (a, b).
The bisection method repeatedly replaces the current interval by one of its two halves. At step k, let

$$x_k = \frac{a_k + b_k}{2}.$$

If $f(a_k)f(x_k) < 0$, retain $[a_k, x_k]$; otherwise retain $[x_k, b_k]$.
A compact implementation is:

```
for k = 1:maxIter
    x = (a+b)/2;

    if abs(f(x)) < tol
        break
    end

    if f(a)*f(x) < 0
        b = x;
    else
        a = x;
    end
end
```

After k bisections, the interval length has been reduced by the factor 2^{-k}. Hence the midpoint error is bounded by

$$|x_k - x^*| \leq \frac{b-a}{2^{k+1}}.$$

The main advantage of bisection is reliability. Its main disadvantage is relatively slow convergence.

9.3 Fixed-Point Iteration

An equation $f(x) = 0$ may be rewritten in an equivalent form

$$x = g(x).$$

A solution x^* satisfying $g(x^*) = x^*$ is called a fixed point of g.
This suggests the iteration

$$x_{n+1} = g(x_n).$$

The MATLAB structure is:

```
x = x0;

for k = 1:maxIter
    xnew = g(x);

    if abs(xnew-x) < tol
        break
    end

    x = xnew;
end
```

The important point is that the same equation may be rewritten in several different fixed-point forms. These forms can have completely different convergence behavior.
For example, the equation $x^2 = 2$ can be written as

$$x = \frac{2}{x}$$

or as

$$x = \frac{1}{2}\left(x + \frac{2}{x}\right).$$

Thus two mathematically equivalent equations may produce very different numerical algorithms.

9.4 Convergence of Fixed-Point Iteration

Suppose g maps an interval $[a, b]$ into itself and is a contraction:

$$|g(x) - g(y)| \le q\,|x - y|\,, \qquad 0 < q < 1.$$

Then g has a unique fixed point in $[a, b]$, and the fixed-point iteration converges to it.
A convenient sufficient condition is

$$|g'(x)| \le q < 1$$

throughout the relevant interval.
The course uses several different rearrangements of the same equation to show that some iterations converge while others diverge. :contentReference[oaicite:2]index=2
Near a fixed point x^*, the magnitude

$$|g'(x^*)|$$

also gives qualitative information about convergence speed. A smaller value corresponds to a flatter graph and usually faster local convergence. :contentReference[oaicite:3]index=3

9.5 Cobweb Diagrams

A cobweb diagram visualizes the iteration

$$x_{n+1} = g(x_n)$$

using the two curves $y = g(x)$ and $y = x$.
Starting from x_0, move vertically to $y = g(x)$, then horizontally to $y = x$, and repeat. The resulting sequence of vertical and horizontal segments forms a cobweb-like pattern. The course uses this representation to compare convergence and divergence of different fixed-point iterations. :contentReference[oaicite:4]index=4
A basic MATLAB implementation is:

```
xgrid = linspace(a,b,400);

plot(xgrid,g(xgrid))
hold on
plot(xgrid,xgrid,'--')

x = x0;

for k = 1:N
    y = g(x);

    plot([x x],[x y])
    plot([x y],[y y])

    x = y;
end

hold off
grid on
```

If the segments move toward the intersection of $y = g(x)$ and $y = x$, the iteration appears convergent. If they move away, the iteration is divergent.

9.6 Newton's Method

Fixed-point iteration raises a natural question: how should g be chosen to obtain rapid convergence?
Starting from $f(x) = 0$, Newton's method uses the tangent line at x_n. The tangent approximation is

$$f(x) \approx f(x_n) + f'(x_n)(x - x_n).$$

Setting the approximation equal to zero gives

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Thus Newton's iteration is

```
x = x0;

for k = 1:maxIter
```

```
    xnew = x - f(x)/df(x);

    if abs(xnew-x) < tol
        break
    end

    x = xnew;
end
```

For example, if

$$f(x) = x^3 + x - 1,$$

then

```
f = @(x) x.^3 + x - 1;
df = @(x) 3*x.^2 + 1;
```

Newton's method is usually much faster than basic fixed-point iteration when the initial value is suitable.

9.7 Limitations of Newton's Method

Newton's method requires evaluation of $f'(x)$ at every iteration. The course emphasizes several situations in which this may be inconvenient:

- the derivative is difficult to compute;
- the derivative is very small near the root;
- the equation has a multiple root;
- the initial value is poorly chosen.

Thus Newton's method is powerful but more sensitive to the structure of the problem than bisection. The course uses these limitations to motivate secant methods. :contentReference[oaicite:5]index=5

9.8 Secant Method

The secant method replaces the derivative in Newton's formula by a difference quotient:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}.$$

Substituting this approximation gives

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.$$

Unlike Newton's method, the secant method does not require an explicit derivative. It requires two starting values.

```
x0 = a;
x1 = b;

for k = 1:maxIter
    f0 = f(x0);
```

```
    f1 = f(x1);

    x2 = x1 - f1*(x1-x0)/(f1-f0);

    if abs(x2-x1) < tol
        break
    end

    x0 = x1;
    x1 = x2;
end
```

The course also introduces a one-point chord method, in which one endpoint is kept fixed, and a two-point secant method, in which the two most recent iterates are used. :contentReference[oaicite:6]index=6

9.9 Nonlinear Systems

For a nonlinear system

$$\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{0},$$

where

$$\boldsymbol{F} = \begin{pmatrix} f_1 \\ \vdots \\ f_n \end{pmatrix},$$

Newton's method generalizes through the Jacobian matrix

$$J(\boldsymbol{x}) = \left[\frac{\partial f_i}{\partial x_j} \right].$$

The Newton update satisfies

$$J(\boldsymbol{x}_k)\boldsymbol{s}_k = -\boldsymbol{F}(\boldsymbol{x}_k),$$

followed by

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{s}_k.$$

The course specifically notes that one should solve this linear system rather than explicitly computing J^{-1}, both for efficiency and numerical stability. :contentReference[oaicite:7]index=7 A generic MATLAB structure is:

```
x = x0;

for k = 1:maxIter
    Fx = F(x);
    Jx = J(x);

    s = -Jx\Fx;
    xnew = x + s;

    if norm(xnew-x) < tol
        break
    end

```

```
    x = xnew;
end
```

9.10 MATLAB Built-In Root Solvers

MATLAB provides several built-in functions for different types of equations. The course highlights `solve`, `fzero`, `fsolve`, and `roots`. :contentReference[oaicite:8]index=8
The symbolic command `solve` was introduced in Chapter 8. For numerical scalar equations, the principal function is `fzero`.

```
f = @(x) x + 2*sin(x).*exp(x) - 1;

x1 = fzero(f,[0,1]);
x2 = fzero(f,0.8);
```

The first syntax provides an interval containing a sign change; the second provides an initial guess.
For a nonlinear system, use `fsolve`.

```
F = @(x) [
    x(1) - sin(x(1)) - 5*x(2);
    2*x(1) + x(2) - cos(x(2))
];

x = fsolve(F,[0 0]);
```

The result may depend on the initial guess when several solutions exist.

9.11 Polynomial Roots

For a polynomial

$$p(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0,$$

store the coefficients in descending powers:

$$A = [a_n, a_{n-1}, \ldots, a_1, a_0].$$

MATLAB returns all real and complex roots by

```
r = roots(A);
```

For example:

```
A = [1 -6 0 5 -3 8];

r = roots(A);
```

The course distinguishes this from `fzero`: `roots` is designed specifically for polynomial coefficient vectors and returns all polynomial roots in the complex plane. :contentReference[oaicite:9]index=9

9.12 Visualization and Initial Values

Before running an iterative solver, plotting $f(x)$ can help determine the number and approximate locations of real roots.

```
x = linspace(a,b,1000);
y = f(x);

plot(x,y)
hold on
yline(0)
grid on
hold off
```

Intersections with the x-axis indicate possible roots.
The course calls this the graphical magnification method: first plot the function, then successively narrow the visible interval around an intersection. This is particularly useful when the numerical method is sensitive to the initial value. :contentReference[oaicite:10]index=10
For an equation

$$g(x) = h(x),$$

one may instead plot both curves and locate their intersections.

```
plot(x,g(x),x,h(x))
grid on
```

9.13 Iteration and Complex Behavior

Repeated iteration does not automatically imply convergence. Even a simple nonlinear map can exhibit qualitatively different long-term behavior when its parameters change. The course explores iterations of a quadratic map and observes convergence, periodic behavior, and increasingly complicated dynamics as the parameter is varied. :contentReference[oaicite:11]index=11
The practical lesson for root finding is that an iteration rule must be analyzed rather than used mechanically. The form of g, its derivative, the initial point, and the region of iteration can all determine whether the method converges.

9.14 Chapter Shorthand

The principal root-finding patterns are:

```
% Bisection
for k = 1:maxIter
    x = (a+b)/2;

    if abs(f(x)) < tol
        break
    end

    if f(a)*f(x) < 0
        b = x;
    else
        a = x;
    end
end

% Fixed-point iteration
```

```
xnew = g(x);

% Newton method
xnew = x - f(x)/df(x);

% Secant method
xnew = x1 ...
    - f(x1)*(x1-x0)/(f(x1)-f(x0));

% Newton method for systems
s = -J(x)\F(x);
xnew = x + s;

% Scalar nonlinear equation
x = fzero(f,[a,b]);
x = fzero(f,x0);

% Nonlinear system
x = fsolve(F,x0);

% Polynomial roots
r = roots(coeff);

% Root visualization
x = linspace(a,b,1000);
plot(x,f(x))
yline(0)
grid on
```

The central idea is that solving a nonlinear equation numerically means constructing a sequence that approaches a root. Different constructions have different requirements and convergence behavior: bisection emphasizes reliability, fixed-point iteration exposes the role of the iteration map, Newton's method exploits derivative information for rapid convergence, and the secant method replaces derivatives by finite differences. MATLAB built-in solvers automate much of this process, but initial values and convergence behavior still require mathematical judgment.

Chapter 10

Interpolation and Data Fitting

10.1 Interpolation and Fitting

Suppose a collection of data points

$$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$$

is available, while the underlying relationship $y = f(x)$ is unknown.
The goal is to construct a simpler function $P(x)$ such that

$$P(x) \approx f(x).$$

Two basic approaches are interpolation and fitting.

Definition 10.1.1. An *interpolating function* satisfies

$$P(x_i) = y_i, \qquad i = 0, 1, \ldots, n.$$

Thus the interpolating curve passes through every given data point.

Definition 10.1.2. A *fitting function* is not required to pass through every data point. Instead, its parameters are selected so that the curve is close to the data according to a specified criterion.

A typical least-squares criterion is

$$J = \sum_{i=1}^{n} \left[P(x_i) - y_i\right]^2.$$

Thus interpolation is appropriate when the observations are regarded as exact, whereas fitting is more natural when the data contain measurement error or random variation.

10.2 Polynomial Interpolation

Given $n + 1$ distinct interpolation nodes $x_0, \ldots, x_n$, there exists a unique polynomial of degree at most n that interpolates the corresponding values.
The Lagrange interpolation polynomial is

$$P_n(x) = \sum_{i=0}^{n} y_i L_i(x),$$

where

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}.$$

Each basis polynomial satisfies

$$L_i(x_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

A direct MATLAB implementation is:

```
function yq = lagrangeInterp(x,y,xq)

n = length(x);
yq = zeros(size(xq));

for i = 1:n
    Li = ones(size(xq));

    for j = 1:n
        if j ~= i
            Li = Li.*(xq-x(j))/(x(i)-x(j));
        end
    end

    yq = yq + y(i)*Li;
end
end
```

Polynomial interpolation is conceptually simple, but high-degree global polynomials may oscillate strongly. For practical data interpolation, piecewise methods are often preferable.

10.3 Piecewise Linear Interpolation

The simplest local interpolation method joins neighboring data points by line segments. For $x_{i-1} \le x \le x_i$,

$$P(x) = \frac{x_i - x}{x_i - x_{i-1}} y_{i-1} + \frac{x - x_{i-1}}{x_i - x_{i-1}} y_i.$$

MATLAB implements one-dimensional interpolation using `interp1`.

```
yi = interp1(x,y,xi);
```

The default method is piecewise linear interpolation. One may specify the method explicitly.

```
yi = interp1(x,y,xi,'linear');
```

Other useful choices include:

```
yi = interp1(x,y,xi,'nearest');
yi = interp1(x,y,xi,'spline');
yi = interp1(x,y,xi,'pchip');
```

The interpolation points `xi` may contain many more points than the original data, producing a smooth visual representation of the underlying relationship.

10.4 Cubic Spline Interpolation

A cubic spline is a piecewise cubic function constructed so that neighboring pieces join smoothly.
On each interval $[x_i, x_{i+1}]$,

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3.$$

The pieces satisfy interpolation conditions together with continuity of the function and its first two derivatives at the interior nodes.

MATLAB provides the direct command

```
yi = spline(x,y,xi);
```

which is equivalent to

```
yi = interp1(x,y,xi,'spline');
```

The piecewise-polynomial representation can also be returned directly.

```
pp = spline(x,y);
```

Then values are evaluated by

```
yi = ppval(pp,xi);
```

MATLAB's `spline` command uses the default not-a-knot endpoint condition.
The structure `pp` stores the interval breaks and the polynomial coefficients for the individual pieces.

```
pp.breaks
pp.coefs
```

10.5 Interpolation Versus Smoothing

Exact interpolation is not always desirable. If the observations contain noise, forcing a curve through every point may reproduce the noise rather than the underlying trend.
A smoothing spline balances smoothness against agreement with the data.
The course introduces

```
pp = csaps(x,y,p);
```

where $0 \leq p \leq 1$ controls the balance between smoothing and interpolation.
A larger value of p forces the curve closer to the observations. In particular, the limiting choices correspond to much stronger smoothing or spline interpolation.
The result can be evaluated by

```
yi = ppval(pp,xi);
```

and plotted by

```
fnplt(pp)
```

The distinction is therefore

$$\text{interpolation} \longrightarrow \text{pass through the data},$$
$$\text{smoothing} \longrightarrow \text{capture the underlying trend}.$$

10.6 Two-Dimensional Interpolation

Suppose values of a function $z = f(x, y)$ are known on a rectangular grid. Interpolation may then be used to estimate the function at intermediate points.
For grid data, MATLAB provides `interp2`.

```
zi = interp2(x,y,z,xi,yi);
```

The interpolation method may be specified explicitly.

```
zi = interp2(x,y,z,xi,yi,'linear');
zi = interp2(x,y,z,xi,yi,'nearest');
zi = interp2(x,y,z,xi,yi,'spline');
zi = interp2(x,y,z,xi,yi,'cubic');
```

For matrix-valued grids, first construct the coordinate matrices.

```
[X,Y] = meshgrid(x,y);
[Xi,Yi] = meshgrid(xi,yi);

Zi = interp2(X,Y,Z,Xi,Yi,'cubic');
```

The interpolated surface can then be visualized by

```
surf(Xi,Yi,Zi)
```

This technique is useful when measurements are available only at discrete spatial locations but a continuous surface representation is required.

10.7 Scattered Data Interpolation

The function `interp2` assumes that the data lie on a regular grid. Experimental observations, however, may occur at irregularly scattered locations.
For scattered data

$$(x_i, y_i, z_i),$$

MATLAB provides

```
Zi = griddata(x,y,z,Xi,Yi);
```

The method may also be specified.

```
Zi = griddata(x,y,z,Xi,Yi,'linear');
Zi = griddata(x,y,z,Xi,Yi,'nearest');
Zi = griddata(x,y,z,Xi,Yi,'cubic');
```

A typical experiment is:

```
[XI,YI] = meshgrid(xmin:h:xmax,ymin:h:ymax);

ZI = griddata(x,y,z,XI,YI,'cubic');

surf(XI,YI,ZI)
```

Thus the basic distinction is:

`interp2`	for gridded data,
`griddata`	for scattered data.

10.8 Least-Squares Fitting

Suppose the model has the form

$$P(x) = c_1\phi_1(x) + \cdots + c_m\phi_m(x),$$

where the basis functions $\phi_1, \ldots, \phi_m$ are known and the coefficients $c_1, \ldots, c_m$ are unknown.

For observations (x_i, y_i), define the residuals

$$r_i = P(x_i) - y_i.$$

The least-squares method chooses the coefficients to minimize

$$J(\boldsymbol{c}) = \sum_{i=1}^{n} r_i^2.$$

Let

$$A_{ij} = \phi_j(x_i).$$

Then the problem becomes

$$\min_{\boldsymbol{c}} \|A\boldsymbol{c} - \boldsymbol{y}\|_2^2.$$

The corresponding normal equations are

$$A^T A\boldsymbol{c} = A^T \boldsymbol{y}.$$

In MATLAB, the least-squares solution is normally obtained directly using

```
c = A\y;
```

rather than explicitly forming the inverse of $A^T A$.

10.9 Polynomial Fitting

For polynomial data fitting,

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0,$$

MATLAB provides `polyfit`.

```
p = polyfit(x,y,n);
```

The returned vector contains the coefficients in descending powers.
The fitted polynomial is evaluated using

```
yi = polyval(p,xi);
```

A typical workflow is:

```
p = polyfit(x,y,2);

xi = linspace(min(x),max(x),200);
yi = polyval(p,xi);

plot(x,y,'o')
hold on
plot(xi,yi)
hold off
```

The polynomial degree should reflect the expected structure of the data. Increasing the degree always provides more flexibility, but does not necessarily produce a better model.

10.10 Linearization of Nonlinear Relations

Some nonlinear models can be transformed into linear relationships.
For an exponential model

$$y = Ae^{bx},$$

taking logarithms gives

$$\log y = \log A + bx.$$

Hence one may perform a linear fit to $(x, \log y)$.

```
p = polyfit(x,log(y),1);

b = p(1);
A = exp(p(2));
```

Similarly, for a power law

$$y = Ax^b,$$

one obtains

$$\log y = \log A + b \log x.$$

Thus:

```
p = polyfit(log(x),log(y),1);

b = p(1);
A = exp(p(2));
```

The course uses this idea in the drug-concentration experiment. Plotting $\log c$ against time produces an approximately linear relationship, suggesting exponential decay of the concentration.

10.11 Nonlinear Least-Squares Fitting

Not every nonlinear model can be conveniently linearized. Suppose the model has the form

$$y = f(\boldsymbol{c}, x),$$

where $\boldsymbol{c}$ is an unknown parameter vector.
The nonlinear least-squares problem is

$$\min_{\boldsymbol{c}} \sum_{i=1}^{n} \left[f(\boldsymbol{c}, x_i) - y_i \right]^2 .$$

MATLAB provides `lsqcurvefit` for this problem.

```
model = @(c,x) ...
    c(1) + c(2)*exp(-0.02*c(3)*x);

c0 = [0.2 0.05 0.05];

c = lsqcurvefit( ...
    model,c0,xdata,ydata);
```

The initial guess `c0` is important because nonlinear least-squares methods are iterative.
The fitted curve can then be evaluated by

```
yfit = model(c,xdata);
```

10.12 Residual-Based Nonlinear Least Squares

The same problem can be written directly in terms of a residual vector

$$\boldsymbol{r}(\boldsymbol{c}) = \begin{pmatrix} f(\boldsymbol{c}, x_1) - y_1 \\ \vdots \\ f(\boldsymbol{c}, x_n) - y_n \end{pmatrix}.$$

Then

$$\min_{\boldsymbol{c}} \|\boldsymbol{r}(\boldsymbol{c})\|_2^2 .$$

MATLAB solves this form using `lsqnonlin`.

```
residual = @(c) ...
    model(c,xdata) - ydata;

c = lsqnonlin(residual,c0);
```

Thus the difference between the two commands is mainly the interface:

`lsqcurvefit` uses model values and observed data,
`lsqnonlin` uses the residual vector directly.

10.13 Constrained Least Squares

Sometimes the fitted coefficients must satisfy additional mathematical or physical restrictions.
A linear least-squares problem may have the form

$$\min_{\boldsymbol{c}} \|A\boldsymbol{c} - \boldsymbol{y}\|_2^2$$

subject to

$$B\boldsymbol{c} \le \boldsymbol{d}, \qquad B_{\text{eq}}\boldsymbol{c} = \boldsymbol{d}_{\text{eq}},$$

and bound constraints

$$\boldsymbol{\ell} \le \boldsymbol{c} \le \boldsymbol{u}.$$

MATLAB provides

```
[c,resnorm] = lsqlin( ...
    A,y,B,d,Beq,deq,lb,ub);
```

Unused constraints may be replaced by empty matrices.

```
[c,resnorm] = lsqlin( ...
    A,y,[],[],[],[],lb,ub);
```

For a nonnegative least-squares problem,

$$\boldsymbol{c} \ge 0,$$

the course uses

```
[c,resnorm] = lsqnonneg(A,y);
```

Constrained fitting is useful when parameter values have physical meanings that make unconstrained solutions unreasonable.

10.14 Model Selection from Data

Fitting is not merely the mechanical execution of `polyfit` or `lsqcurvefit`. The form of the model should first be suggested by the data or by the underlying mechanism.
A practical workflow is

$$\text{plot the observations} \longrightarrow \text{identify a plausible model}$$
$$\text{estimate parameters} \longrightarrow \text{inspect residuals and fit.}$$

For example, if a plot of y against x is curved but $\log y$ against x is approximately linear, an exponential model is a natural candidate.
Likewise, a log–log plot that is approximately linear suggests a power-law relationship.
Visualization therefore remains an important part of data fitting.

10.15 Application Pattern: Drug Concentration

The course uses measured blood-drug concentration data to illustrate model identification from experimental observations.
If the concentration follows approximately

$$c(t) = c_0 e^{-kt},$$

then

$$\log c(t) = \log c_0 - kt.$$

Thus the nonlinear relation becomes a linear fitting problem.

```
p = polyfit(t,log(c),1);

k = -p(1);
c0 = exp(p(2));

cfit = c0*exp(-k*t);
```

The experiment illustrates the full modeling process:

$$\text{experimental data} \longrightarrow \text{transform and visualize}$$
$$\text{identify model} \longrightarrow \text{fit parameters} \longrightarrow \text{interpret the model.}$$

The same principle applies far beyond this particular application.

10.16 Chapter Shorthand

The most useful interpolation and fitting commands are:

```
% One-dimensional interpolation
yi = interp1(x,y,xi);
yi = interp1(x,y,xi,'linear');
yi = interp1(x,y,xi,'nearest');
yi = interp1(x,y,xi,'spline');
yi = interp1(x,y,xi,'pchip');

% Cubic spline
yi = spline(x,y,xi);
pp = spline(x,y);
```

```
yi = ppval(pp,xi);

% Smoothing spline
pp = csaps(x,y,p);
yi = ppval(pp,xi);

% Two-dimensional interpolation
Zi = interp2(X,Y,Z,Xi,Yi);
Zi = interp2( ...
    X,Y,Z,Xi,Yi,'cubic');

% Scattered interpolation
Zi = griddata(x,y,z,Xi,Yi);
Zi = griddata( ...
    x,y,z,Xi,Yi,'cubic');

% Polynomial fitting
p = polyfit(x,y,n);
yi = polyval(p,xi);

% Linear least squares
c = A\y;

% Nonlinear curve fitting
c = lsqcurvefit( ...
    model,c0,xdata,ydata);

% Residual nonlinear least squares
c = lsqnonlin(residual,c0);

% Nonnegative least squares
[c,resnorm] = lsqnonneg(A,y);

% Constrained linear least squares
[c,resnorm] = lsqlin( ...
    A,y,B,d,Beq,deq,lb,ub);

% Exponential fitting by linearization
p = polyfit(x,log(y),1);
b = p(1);
A0 = exp(p(2));

% Power-law fitting
p = polyfit(log(x),log(y),1);
b = p(1);
A0 = exp(p(2));
```

The essential distinction is that interpolation reconstructs values between known data points while requiring exact agreement at those points, whereas fitting estimates an underlying relationship without requiring exact agreement. Piecewise interpolation is generally preferable for local reconstruction, while least-squares fitting is appropriate when

the data contain error and the objective is to identify a global trend or model.

Chapter 11

Numerical Integration

11.1 From Definite Integrals to Numerical Quadrature

Consider the definite integral

$$I = \int_a^b f(x)\,dx.$$

If an antiderivative is unavailable or difficult to obtain, the integral can be approximated directly from sampled values of f.
Partition the interval as

$$a = x_0 < x_1 < \cdots < x_n = b.$$

If $\xi_i \in [x_{i-1}, x_i]$, then the Riemann-sum approximation is

$$I \approx \sum_{i=1}^{n} f(\xi_i)(x_i - x_{i-1}).$$

For equally spaced nodes, let

$$h = \frac{b-a}{n}, \qquad x_i = a + ih.$$

Different choices of sampling points and local approximations lead to different quadrature rules.
The common computational pattern is

$$\text{partition the interval} \longrightarrow \text{sample the function}$$
$$\text{form a weighted sum} \longrightarrow \text{approximate the integral.}$$

11.2 Rectangle Rules

The left rectangle rule uses the left endpoint of every subinterval:

$$I \approx h \sum_{i=0}^{n-1} f(x_i).$$

A MATLAB implementation is:

```
h = (b-a)/n;
x = a:h:b;

Ileft = h*sum(f(x(1:end-1)));
```

The right rectangle rule uses the right endpoints:

$$I \approx h \sum_{i=1}^{n} f(x_i).$$

```
Iright = h*sum(f(x(2:end)));
```

The midpoint rule instead uses

$$m_i = \frac{x_{i-1} + x_i}{2},$$

giving

$$I \approx h \sum_{i=1}^{n} f(m_i).$$

```
h = (b-a)/n;

xmid = a+h/2:h:b-h/2;

Imid = h*sum(f(xmid));
```

The midpoint rule generally provides a substantial improvement over the one-sided rectangle rules because the local sampling point is more balanced.

11.3 Composite Trapezoidal Rule

Instead of approximating f by a constant on each subinterval, the trapezoidal rule replaces it by the straight line joining the endpoint values.
On $[x_{i-1}, x_i]$,

$$\int_{x_{i-1}}^{x_i} f(x)\,dx \approx \frac{h}{2}\left[f(x_{i-1}) + f(x_i)\right].$$

Summing over all intervals gives

$$T_n = \frac{h}{2}\left[f(x_0) + 2\sum_{i=1}^{n-1} f(x_i) + f(x_n)\right].$$

A direct MATLAB implementation is:

```
h = (b-a)/n;
x = a:h:b;
y = f(x);

Itrap = h*( ...
    0.5*y(1) ...
    + sum(y(2:end-1)) ...
    + 0.5*y(end));
```

MATLAB also provides the built-in command

```
Itrap = trapz(x,y);
```

which applies the trapezoidal rule directly to sampled data.

11.4 Composite Simpson Rule

Simpson's rule approximates f locally by quadratic polynomials.
For two consecutive subintervals,

$$\int_{x_{2j}}^{x_{2j+2}} f(x)\,dx \approx \frac{h}{3}\left[f(x_{2j}) + 4f(x_{2j+1}) + f(x_{2j+2})\right].$$

Therefore, when n is even,

$$S_n = \frac{h}{3}\left[f(x_0) + 4\sum_{\substack{i=1\\ i \text{ odd}}}^{n-1} f(x_i) + 2\sum_{\substack{i=2\\ i \text{ even}}}^{n-2} f(x_i) + f(x_n)\right].$$

A MATLAB implementation is:

```
h = (b-a)/n;
x = a:h:b;
y = f(x);

Isimpson = h/3*( ...
    y(1) ...
    + 4*sum(y(2:2:end-1)) ...
    + 2*sum(y(3:2:end-2)) ...
    + y(end));
```

The number of subintervals n must be even.
In the course experiments, the same integral is computed using rectangle, midpoint, trapezoidal, and Simpson rules. Increasing n improves all approximations, while Simpson's rule gives much smaller errors in the illustrated comparison.

11.5 Interpolation Viewpoint

Numerical quadrature can also be interpreted through interpolation.
On each subinterval, replace f by a simpler polynomial P and compute

$$\int f(x)\,dx \approx \int P(x)\,dx.$$

Different polynomial approximations produce different rules:

$$\begin{aligned}&\text{constant approximation} \longrightarrow \text{rectangle rules},\\ &\text{linear interpolation} \longrightarrow \text{trapezoidal rule},\\ &\text{quadratic interpolation} \longrightarrow \text{Simpson's rule}.\end{aligned}$$

Thus numerical integration and interpolation are closely related. The quadrature formula replaces a difficult function by a simple local approximation whose integral can be evaluated explicitly.

11.6 Convergence Experiments

A basic numerical experiment compares the approximation

$$I_n$$

with an exact value I while increasing the number of subintervals.
Define

$$E_n = |I_n - I|\,.$$

A typical MATLAB structure is:

```
nList = [10 20 40 80 160];

for j = 1:length(nList)
    n = nList(j);

    Iapprox(j) = quadrature(f,a,b,n);
    err(j) = abs(Iapprox(j)-Iexact);
end
```

The errors can then be examined on logarithmic axes.

```
loglog(nList,err,'o-')
grid on
```

Such experiments reveal whether refinement of the partition actually improves the approximation and allow different quadrature rules to be compared directly.

11.7 The `trapz` Function

The function `trapz` applies the trapezoidal rule to discrete data.
If only the data vector is supplied,

```
I = trapz(y);
```

MATLAB treats the independent variable as the element index.
If the sampling points are known explicitly, use

```
I = trapz(x,y);
```

This form is preferable when the spacing is not 1, and it also supports nonuniformly spaced data.
For uniformly spaced data with spacing h,

```
I = h*trapz(y);
```

is equivalent to supplying the corresponding coordinate vector.

11.8 Adaptive Quadrature

A fixed-grid method uses the same step size throughout the integration interval. This may be inefficient if the function is smooth in some regions but changes rapidly in others. An adaptive quadrature method repeatedly estimates the local error and refines the partition only where additional resolution is needed.
The basic idea is

$$\text{approximate on a coarse interval} \longrightarrow \text{estimate local error}$$
$$\text{refine if necessary} \longrightarrow \text{repeat until tolerance is met.}$$

This allows computational effort to be concentrated in difficult parts of the integrand.

11.9 The `quad` Function

The course introduces `quad` as an adaptive recursive Simpson method.
Its basic form is

```
q = quad(fun,a,b);
```

A tolerance may be specified by

```
q = quad(fun,a,b,tol);
```

For example:

```
f = @(x) 4./(1+x.^2);

q = quad(f,0,1);
```

Since

$$\int_0^1 \frac{4}{1+x^2}\,dx = \pi,$$

this provides another numerical experiment for approximating π.
The course gives the default tolerance of `quad` as 10^{-6}.

11.10 The `quadl` Function

The function `quadl` uses an adaptive Lobatto quadrature method.
Its syntax is similar to `quad`.

```
q = quadl(fun,a,b);
q = quadl(fun,a,b,tol);
```

The course describes `quadl` as particularly suitable when higher accuracy is desired and the integrand is relatively smooth.
For an integrand containing an additional parameter, an anonymous function can be used.

```
f = @(x,c) 1./(x.^3-2*x-c);

q = quadl(@(x) f(x,10),0,2);
```

11.11 The `quadgk` Function

The function `quadgk` uses adaptive Gauss–Kronrod quadrature.
The course emphasizes that it is useful for problems involving high accuracy, oscillation, infinite intervals, endpoint singularities, or more complicated integration paths.
The basic syntax is

```
q = quadgk(fun,a,b);
```

An error estimate may also be returned.

```
[q,errbnd] = quadgk(fun,a,b);
```

Absolute and relative tolerances can be specified.

```
[q,errbnd] = quadgk( ...
    fun,a,b, ...
    'AbsTol',1e-10, ...
    'RelTol',1e-8);
```

Improper integrals are allowed when the integral converges.

```
f = @(x) exp(-x.^2).*(log(x)).^2;

q = quadgk(f,0,Inf);
```

For a function with internal difficult points, the course also introduces the `Waypoints` option.

11.12 Double Integrals

For a function $f(x,y)$ defined on a region D,

$$I = \iint_D f(x,y)\,dA.$$

Geometrically, when $f \geq 0$, this represents the volume below the surface $z = f(x, y)$ and above the region D.
For a rectangular region

$$D = [a, b] \times [c, d],$$

divide the region into small rectangles and choose one sample point in each. Then

$$I \approx \sum_{i,j} f(x_i^*, y_j^*) \Delta x \, \Delta y.$$

A simple midpoint implementation is:

```
dx = (b-a)/m;
dy = (d-c)/n;

x = a+dx/2:dx:b-dx/2;
y = c+dy/2:dy:d-dy/2;

[X,Y] = meshgrid(x,y);

I = sum(sum(f(X,Y)))*dx*dy;
```

This is the two-dimensional analogue of the midpoint rule.

11.13 Nonrectangular Regions

A two-dimensional integration region may be described as

$$D = \{(x, y) : a \leq x \leq b,\ g_1(x) \leq y \leq g_2(x)\}.$$

Then

$$\iint_D f(x, y) \, dA = \int_a^b \left[\int_{g_1(x)}^{g_2(x)} f(x, y) \, dy \right] dx.$$

The numerical computation may therefore be regarded as repeated one-dimensional integration.
The same principle applies when the region is more naturally written in the opposite order:

$$c \leq y \leq d, \qquad h_1(y) \leq x \leq h_2(y).$$

The course treats both rectangular regions and regions described by variable upper and lower boundaries.

11.14 MATLAB Multidimensional Quadrature

For a rectangular region, the course introduces `dblquad`.

```
q = dblquad( ...
    fun,xmin,xmax,ymin,ymax);
```

A tolerance may be specified.

```
q = dblquad( ...
    fun,xmin,xmax,ymin,ymax,tol);
```

The integration method may also be supplied.

```
q = dblquad( ...
    fun,xmin,xmax,ymin,ymax, ...
    1e-8,@quadl);
```

For a rectangular box in three dimensions, the course uses `triplequad`.

```
q = triplequad( ...
    fun,xmin,xmax, ...
    ymin,ymax, ...
    zmin,zmax);
```

Thus the same quadrature idea extends from intervals to areas and volumes.

11.15 Choosing a Numerical Integration Method

Different numerical integration methods are useful in different settings.
For discrete experimental data, `trapz` is natural because no explicit function formula is required.
For a smooth function known analytically, adaptive methods reduce the need to choose a sufficiently fine mesh manually.
For difficult improper or oscillatory integrals, the course emphasizes `quadgk`.
For teaching and algorithmic understanding, implementing the rectangle, trapezoidal, and Simpson rules directly is valuable because it reveals how quadrature formulas are constructed.
Thus the method should depend on the available information and the structure of the integrand.

11.16 Chapter Shorthand

The principal formulas and MATLAB patterns are:

```
% Grid
h = (b-a)/n;
x = a:h:b;
y = f(x);

% Left rectangle
I = h*sum(y(1:end-1));

% Right rectangle
I = h*sum(y(2:end));

% Midpoint rule
xm = a+h/2:h:b-h/2;
I = h*sum(f(xm));

% Trapezoidal rule
I = h*( ...
    0.5*y(1) ...
    + sum(y(2:end-1)) ...
    + 0.5*y(end));

% Built-in trapezoidal rule
```

```
I = trapz(x,y);

% Composite Simpson rule
I = h/3*( ...
    y(1) ...
    + 4*sum(y(2:2:end-1)) ...
    + 2*sum(y(3:2:end-2)) ...
    + y(end));

% Adaptive Simpson quadrature
q = quad(fun,a,b);
q = quad(fun,a,b,tol);

% Adaptive Lobatto quadrature
q = quadl(fun,a,b);
q = quadl(fun,a,b,tol);

% Adaptive Gauss--Kronrod quadrature
q = quadgk(fun,a,b);

[q,errbnd] = quadgk( ...
    fun,a,b, ...
    'AbsTol',1e-10, ...
    'RelTol',1e-8);

% Improper integral
q = quadgk(fun,0,Inf);

% Double integral on a rectangle
q = dblquad( ...
    fun,xmin,xmax,ymin,ymax);

% Triple integral on a box
q = triplequad( ...
    fun,xmin,xmax, ...
    ymin,ymax, ...
    zmin,zmax);
```

The central idea of numerical integration is to replace the integral of a complicated function by a weighted sum of sampled function values. Rectangle, trapezoidal, and Simpson rules differ in the local approximation used, while adaptive quadrature improves efficiency by refining the partition only where necessary. For multidimensional problems, the same principle is applied over small areas or volumes rather than subintervals.

Chapter 12

Optimization

12.1 Optimization Models

An optimization problem seeks the best feasible choice according to a specified objective. A mathematical optimization model contains three basic components:

- decision variables;
- an objective function;
- constraints.

Let

$$\boldsymbol{x} = (x_1, x_2, \ldots, x_n)^T$$

denote the decision vector. A general minimization problem can be written as

$$\min_{\boldsymbol{x}} f(\boldsymbol{x})$$

subject to

$$h_i(\boldsymbol{x}) = 0, \qquad g_j(\boldsymbol{x}) \leq 0.$$

A maximization problem can always be converted to minimization through

$$\max f(\boldsymbol{x}) \quad \Longleftrightarrow \quad \min[-f(\boldsymbol{x})].$$

Thus the general modeling workflow is

$$\begin{aligned} &\text{identify decision variables} \longrightarrow \text{construct constraints} \\ &\text{construct objective} \longrightarrow \text{solve and interpret.} \end{aligned}$$

12.2 Local and Global Optima

A point $\boldsymbol{x}^*$ is a local minimizer if

$$f(\boldsymbol{x}^*) \leq f(\boldsymbol{x})$$

for all feasible points sufficiently close to $\boldsymbol{x}^*$.
It is a global minimizer if the same inequality holds for every feasible point.
This distinction is essential in nonlinear optimization. An iterative method may converge to a nearby local minimum without locating the global minimum.
For a one-dimensional function, visualization can help reveal multiple local extrema.

```
f = @(x) x.*sin(3*x);

fplot(f,[0 6])
grid on
```

The course explicitly emphasizes that a nonlinear optimization routine cannot in general guarantee that the returned local optimum is globally optimal.

12.3 First-Order Optimality

For a differentiable unconstrained function

$$f : \mathbb{R}^n \to \mathbb{R},$$

a necessary condition for an interior local optimum is

$$\nabla f(\boldsymbol{x}^*) = \mathbf{0}.$$

For two variables,

$$\nabla f = \begin{pmatrix} f_x \\ f_y \end{pmatrix}.$$

The Hessian matrix is

$$H_f = \begin{pmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{pmatrix}.$$

At a stationary point, a positive definite Hessian indicates a strict local minimum, while a negative definite Hessian indicates a strict local maximum.
Symbolically, MATLAB can compute these quantities by

```
syms x y

f = fexpr;

g = jacobian(f,[x y]);
H = hessian(f,[x y]);
```

This connects symbolic differentiation with numerical optimization.

12.4 Gradient Descent

For unconstrained minimization, the negative gradient gives the direction of steepest local decrease.
Starting from $\boldsymbol{x}_k$, gradient descent uses

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \alpha_k \nabla f(\boldsymbol{x}_k),$$

where $\alpha_k > 0$ is the step size.
A basic MATLAB implementation is:

```
x = x0;

for k = 1:maxIter
    g = gradf(x);

    if norm(g) < tol
        break
    end

    x = x - alpha*g;
end
```

The course illustrates this idea through the “blind man descending a mountain” experiment: the current point represents the location on the terrain, while the negative gradient indicates the steepest downhill direction. :contentReference[oaicite:1]index=1
The step size is crucial. If it is too large, the iteration may overshoot; if it is too small, convergence may be unnecessarily slow.

12.5 One-Dimensional Unconstrained Optimization

For a scalar function on a bounded interval,

$$\min_{a \le x \le b} f(x),$$

MATLAB provides `fminbnd`.

```
[xmin,fval] = fminbnd(f,a,b);
```

For example:

```
f = @(x) x.*sin(3*x);

[xmin,fval] = fminbnd(f,0,6);
```

The routine returns a local minimizer within the specified interval.
For a maximum, minimize the negative function.

```
[xmax,fneg] = fminbnd( ...
    @(x) -f(x),a,b);

fmax = -fneg;
```

12.6 Unconstrained Multivariable Optimization

For

$$\min_{\boldsymbol{x} \in \mathbb{R}^n} f(\boldsymbol{x}),$$

the course introduces `fminunc`.

```
[x,fval] = fminunc(fun,x0);
```

Here `x0` is an initial guess. The result may depend on this starting point. The course also allows optimization options and additional diagnostic outputs such as `exitflag`. :contentReference[oaicite:2]index=2
Another function introduced in the course is `fminsearch`.

```
[x,fval] = fminsearch(fun,x0);
```

The course uses it for unconstrained multivariable problems and emphasizes that the initial point can strongly affect the result. :contentReference[oaicite:3]index=3
For example, a location problem may minimize the total distance from a point (x, y) to several prescribed locations:

```
fun = @(x) sum( ...
    sqrt((x(1)-xi).^2 + ...
    (x(2)-yi).^2));

[x,fval] = fminsearch(fun,[5 3]);
```

This converts a geometric location problem directly into an unconstrained optimization model.

12.7 Linear Programming

A linear programming problem has a linear objective and linear constraints.
MATLAB uses the standard minimization form

$$\min_{\boldsymbol{x}} \boldsymbol{c}^T \boldsymbol{x}$$

subject to

$$A\boldsymbol{x} \le \boldsymbol{b},$$

$$A_{\text{eq}}\boldsymbol{x} = \boldsymbol{b}_{\text{eq}},$$

and

$$\boldsymbol{\ell} \le \boldsymbol{x} \le \boldsymbol{u}.$$

The basic MATLAB command is

```
[x,fval] = linprog( ...
    c,A,b,Aeq,beq,lb,ub);
```

Unused constraints are represented by empty matrices.

```
[x,fval] = linprog( ...
    c,A,b,[],[],lb,ub);
```

The course emphasizes that `linprog` requires inequality constraints to be written in the form

$$A\boldsymbol{x} \le \boldsymbol{b}.$$

Therefore a constraint such as

$$a^T \boldsymbol{x} \ge d$$

must first be rewritten as

$$-a^T \boldsymbol{x} \le -d.$$

:contentReference[oaicite:4]index=4

12.8 Linear Programming Modeling

The difficult part of a linear programming problem is often not the numerical solution but the construction of the model.
A typical resource-allocation model has the structure

$$\max \sum_{i=1}^{n} p_i x_i$$

subject to

$$\sum_{i=1}^{n} a_{ji} x_i \le b_j, \qquad j = 1, \dots, m,$$

and

$$x_i \ge 0.$$

Here x_i represents the amount assigned to activity i, p_i is its profit or benefit, and a_{ji} measures consumption of resource j.
Since `linprog` minimizes, write

```
c = -profit;

[x,fval] = linprog( ...
    c,A,b,Aeq,beq,lb,ub);

maxProfit = -fval;
```

The course applies this pattern to production, feeding, cutting-stock, and investment problems. In each case, the physical or economic restrictions are translated into linear inequalities before MATLAB is called.

12.9 Quadratic Programming

A quadratic programming problem has a quadratic objective and linear constraints. The standard form used by MATLAB is

$$\min_{\boldsymbol{x}} \left(\frac{1}{2} \boldsymbol{x}^T H \boldsymbol{x} + \boldsymbol{f}^T \boldsymbol{x} \right)$$

subject to

$$A\boldsymbol{x} \le \boldsymbol{b},$$

$$A_{\mathrm{eq}} \boldsymbol{x} = \boldsymbol{b}_{\mathrm{eq}},$$

and

$$\boldsymbol{\ell} \le \boldsymbol{x} \le \boldsymbol{u}.$$

The MATLAB command is

```
[x,fval] = quadprog( ...
    H,f,A,b,Aeq,beq,lb,ub);
```

For example:

```
H = [2 1 1;
     1 2 0;
     1 0 2];

f = [-5; -3; -6];

Aeq = [5 4 7];
beq = 22;

lb = zeros(3,1);

[x,fval] = quadprog( ...
    H,f,[],[],Aeq,beq,lb,[]);
```

The course explicitly uses `quadprog` for this standard quadratic model. :contentReference[oaicite:5]index=5

12.10 General Nonlinear Programming

The general constrained nonlinear optimization model in the course is

$$\min_{\boldsymbol{x}} f(\boldsymbol{x})$$

subject to

$$A\boldsymbol{x} \le \boldsymbol{b},$$
$$A_{\mathrm{eq}}\boldsymbol{x} = \boldsymbol{b}_{\mathrm{eq}},$$
$$\boldsymbol{c}(\boldsymbol{x}) \le \boldsymbol{0},$$
$$\boldsymbol{c}_{\mathrm{eq}}(\boldsymbol{x}) = \boldsymbol{0},$$

and

$$\boldsymbol{\ell} \le \boldsymbol{x} \le \boldsymbol{u}.$$

MATLAB solves this form using `fmincon`.

```
[x,fval] = fmincon( ...
    fun,x0,A,b,Aeq,beq, ...
    lb,ub,nonlcon);
```

The standard model used by the course contains both linear and nonlinear equality and inequality constraints. :contentReference[oaicite:6]index=6

12.11 Nonlinear Constraint Functions

The nonlinear constraint function must return two outputs:

```
function [c,ceq] = nonlcon(x)

c = [
    g1(x);
    g2(x)
];

ceq = [
    h1(x);
    h2(x)
];

end
```

MATLAB interprets these as

$$\boldsymbol{c}(\boldsymbol{x}) \le 0$$

and

$$\boldsymbol{c}_{\mathrm{eq}}(\boldsymbol{x}) = 0.$$

If there are no nonlinear equality constraints, use

```
ceq = [];
```

A complete constrained problem therefore has the pattern

```
x0 = initialGuess;

[x,fval] = fmincon( ...
    @objfun,x0, ...
    A,b,Aeq,beq, ...
    lb,ub,@nonlcon);
```

This is the principal MATLAB template for general nonlinear optimization. :contentReference[oaicite:7]index=7

12.12 Initial Values and Local Minima

Nonlinear optimization routines are generally iterative. Consequently, the initial point $\boldsymbol{x}_0$ can influence both convergence and the final solution.
The course gives an example in which `fminsearch` fails for one nearby initial point because of numerical overflow, while another starting point allows the method to locate a minimum. :contentReference[oaicite:8]index=8
This leads to two practical questions:

How should a good initial point be chosen?

and

How can a local minimum be distinguished from a global minimum?

Useful experimental strategies include plotting low-dimensional objective functions, trying several initial values, and comparing the resulting solutions.
A numerical optimization result should therefore not automatically be interpreted as a global optimum.

12.13 Integer Programming

In some applications, decision variables must take integer values. Examples include the number of machines, vehicles, items, or selected projects.
A mixed-integer linear model has the form

$$\min_{\boldsymbol{x}} \boldsymbol{f}^T \boldsymbol{x}$$

subject to linear constraints, with selected components of $\boldsymbol{x}$ restricted to integers.
MATLAB uses

```
[x,fval] = intlinprog( ...
    f,intcon,A,b,Aeq,beq,lb,ub);
```

Here `intcon` contains the indices of the integer-valued variables.
For example, if x_1 and x_3 must be integers,

```
intcon = [1 3];
```

The course introduces integer programming through problems such as the knapsack problem, where a continuous linear-programming solution would not necessarily correspond to a feasible physical decision. :contentReference[oaicite:9]index=9

12.14 Heuristic and Intelligent Optimization

Classical optimization methods exploit mathematical structure such as gradients, linearity, or convexity. For complicated objective functions, another possibility is to use heuristic search.
The later course material introduces the genetic algorithm as a method that simulates natural evolution to search for good solutions. It does not require a detailed analytic description of the objective landscape. :contentReference[oaicite:10]index=10
The MATLAB genetic algorithm function is

```
[x,fval] = ga(fun,nvars);
```

Linear and nonlinear constraints can also be included.

```
[x,fval] = ga( ...
    fun,nvars,A,b,Aeq,beq, ...
    lb,ub,nonlcon,intcon);
```

Because the algorithm contains random search mechanisms, repeated runs may produce slightly different results. The course explicitly observes this randomness in its genetic-algorithm experiment. :contentReference[oaicite:11]index=11
The role of such methods in these notes is supplementary: they provide a way to explore difficult optimization landscapes when standard local methods are sensitive to initial values or become trapped in local extrema.

12.15 Choosing an Optimization Solver

The optimization model should determine the MATLAB solver.

bounded scalar problem $\longrightarrow$ `fminbnd`,
unconstrained multivariable problem $\longrightarrow$ `fminunc/fminsearch`,
linear programming $\longrightarrow$ `linprog`,
quadratic programming $\longrightarrow$ `quadprog`,
general constrained nonlinear problem $\longrightarrow$ `fmincon`.

Integer restrictions lead to `intlinprog`, while heuristic methods such as `ga` provide another option for difficult search problems.
The important step is therefore not to choose a MATLAB command first. One should first identify the mathematical structure of the optimization model.

12.16 Chapter Shorthand

The principal optimization commands are:

```
% One-dimensional bounded minimization
[x,fval] = fminbnd(fun,a,b);

% Unconstrained multivariable optimization
[x,fval] = fminunc(fun,x0);
[x,fval] = fminsearch(fun,x0);

% Convert maximum to minimum
[x,fneg] = fminunc( ...
    @(x) -fun(x),x0);

fmax = -fneg;

% Linear programming
[x,fval] = linprog( ...
    c,A,b,Aeq,beq,lb,ub);

% Quadratic programming
[x,fval] = quadprog( ...
    H,f,A,b,Aeq,beq,lb,ub);

% General nonlinear programming
```

```
[x,fval] = fmincon( ...
    fun,x0,A,b,Aeq,beq, ...
    lb,ub,nonlcon);

% Nonlinear constraints
function [c,ceq] = nonlcon(x)

c = [
    g1(x);
    g2(x)
];

ceq = [
    h1(x)
];

end

% Integer linear programming
[x,fval] = intlinprog( ...
    f,intcon,A,b,Aeq,beq,lb,ub);

% Genetic algorithm
[x,fval] = ga(fun,nvars);
```

The central principle of optimization is to separate modeling from numerical solution. First determine the decision variables, objective function, and constraints. Then identify whether the model is unconstrained, linear, quadratic, nonlinear, or integer-valued, and finally choose the corresponding solver. For nonlinear problems, the returned solution may depend on the initial point and may represent only a local optimum, so numerical results should be checked against the geometry and structure of the model.

Chapter 13

Differential Equations

13.1 Differential Equations as Dynamic Models

Differential equations describe systems through their rates of change. They arise naturally when the current state of a system determines how that state changes with time.
Typical applications include motion, population growth, chemical reactions, inventory systems, and control problems.
A first-order initial value problem has the form

$$\begin{cases} y'(t) = f(t, y), \\ y(t_0) = y_0. \end{cases}$$

When an explicit analytic solution is unavailable, numerical methods construct approximations

$$y_n \approx y(t_n)$$

at discrete time points

$$t_n = t_0 + nh.$$

Thus the numerical problem has the general structure

$$\begin{aligned} &\text{differential equation} \longrightarrow \text{discrete update rule} \\ &\text{initial condition} \longrightarrow \text{numerical trajectory.} \end{aligned}$$

13.2 Forward Euler Method

From

$$y'(t_n) \approx \frac{y_{n+1} - y_n}{h},$$

substitution into $y' = f(t, y)$ gives

$$y_{n+1} = y_n + hf(t_n, y_n).$$

This is the *Forward Euler Method*, also called the explicit Euler method.
A direct MATLAB implementation is:

```
t = t0:h:T;
N = length(t);

y = zeros(1,N);
y(1) = y0;

for n = 1:N-1
    y(n+1) = y(n) ...
        + h*f(t(n),y(n));
end
```

The method uses only information at the current point, so each new value can be computed explicitly.
The course emphasizes that Forward Euler is only first-order accurate. Therefore a small step size may be required to obtain acceptable accuracy.

13.3 Backward Euler Method

The *Backward Euler Method* evaluates the derivative at the new time level:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}).$$

Unlike Forward Euler, the unknown y_{n+1} appears inside f. Therefore the method is implicit.
If f is nonlinear, each step may require solving a nonlinear equation.
For example, one may define

$$F(z) = z - y_n - hf(t_{n+1}, z)$$

and solve

$$F(z) = 0.$$

A MATLAB pattern is:

```
for n = 1:N-1
    tn = t(n+1);
    yn = y(n);

    F = @(z) ...
        z - yn - h*f(tn,z);

    y(n+1) = fzero(F,yn);
end
```

Backward Euler is computationally more expensive, but the course notes that its stability is generally better than that of Forward Euler.

13.4 Trapezoidal and Crank–Nicolson Method

A more symmetric approximation averages the slopes at the two endpoints:

$$y_{n+1} = y_n + \frac{h}{2}\left[f(t_n, y_n) + f(t_{n+1}, y_{n+1})\right].$$

This is the trapezoidal or Crank–Nicolson formula.
It is a second-order method and is generally more accurate than either Forward or Backward Euler for the same step size.
However, y_{n+1} appears implicitly, so nonlinear problems again require the solution of an algebraic equation at each step.
A generic implementation is:

```
for n = 1:N-1
    tn = t(n);
    yn = y(n);

    F = @(z) ...
        z - yn ...
        - h/2*( ...
        f(tn,yn) ...
        + f(tn+h,z));

    y(n+1) = fzero(F,yn);
end
```

13.5 Improved Euler Method

The implicit trapezoidal formula can be converted into an explicit predictor–corrector scheme.
First predict using Forward Euler:

$$\widetilde{y}_{n+1} = y_n + hf(t_n, y_n).$$

Then correct using the average slope:

$$y_{n+1} = y_n + \frac{h}{2}\left[f(t_n, y_n) + f(t_{n+1}, \widetilde{y}_{n+1})\right].$$

This is commonly called the *Improved Euler Method*.

```
for n = 1:N-1
    tn = t(n);
    yn = y(n);

    yp = yn + h*f(tn,yn);

    y(n+1) = yn ...
        + h/2*( ...
        f(tn,yn) ...
        + f(tn+h,yp));
end
```

The method retains the simplicity of an explicit scheme while obtaining second-order accuracy.

13.6 Runge–Kutta Methods

Runge–Kutta methods improve accuracy by evaluating several slopes within one time step.
A general two-stage form introduced in the course is

$$y_{n+1} = y_n + h\left[(1-\alpha)k_1 + \alpha k_2\right],$$

where

$$k_1 = f(t_n, y_n),$$

and

$$k_2 = f(t_n + \beta h,\, y_n + \beta h k_1).$$

Suitable choices of the parameters produce a second-order method.
The main idea is to combine several slope estimates rather than relying on a single derivative evaluation.

13.7 Classical Fourth-Order Runge–Kutta Method

The classical fourth-order Runge–Kutta method is one of the most widely used explicit ODE methods.
Define

$$k_1 = f(t_n, y_n),$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right),$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right),$$

and

$$k_4 = f(t_n + h, y_n + hk_3).$$

Then

$$y_{n+1} = y_n + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).$$

A MATLAB implementation is:

```
for n = 1:N-1
    tn = t(n);
    yn = y(n);

    k1 = f(tn,yn);

    k2 = f( ...
        tn+h/2, ...
        yn+h*k1/2);

    k3 = f( ...
        tn+h/2, ...
        yn+h*k2/2);

    k4 = f( ...
        tn+h, ...
        yn+h*k3);

    y(n+1) = yn ...
        + h*( ...
        k1 + 2*k2 ...
        + 2*k3 + k4)/6;
end
```

The course includes an experiment comparing the fourth-order Runge–Kutta approximation with a known analytic solution.

13.8 Comparing Numerical Methods

When an exact solution $y(t)$ is available, numerical methods can be compared through

$$E_n = |y_n - y(t_n)|.$$

A typical experiment computes several numerical solutions on the same grid.

```
yEuler = eulerMethod( ...
    f,t0,T,y0,h);

yImproved = improvedEuler( ...
    f,t0,T,y0,h);

```

```
yRK4 = rk4Method( ...
    f,t0,T,y0,h);
```

They can then be compared with the exact solution.

```
yExact = exact(t);

plot(t,yExact)
hold on

plot(t,yEuler,'--')
plot(t,yImproved,'-.')
plot(t,yRK4,':')

hold off
legend( ...
    'Exact', ...
    'Euler', ...
    'Improved Euler', ...
    'RK4')
```

The experiment reveals the trade-off between computational effort and accuracy.

13.9 From Higher-Order Equations to Systems

MATLAB numerical ODE solvers are naturally formulated for first-order systems.
A higher-order equation can therefore be converted into a system.
For example, consider

$$y'' = F(t, y, y').$$

Set

$$x_1 = y, \qquad x_2 = y'.$$

Then

$$x_1' = x_2,$$

and

$$x_2' = F(t, x_1, x_2).$$

Thus the second-order equation becomes

$$\boldsymbol{x}' = \begin{pmatrix} x_2 \\ F(t, x_1, x_2) \end{pmatrix}.$$

The corresponding MATLAB function is:

```
function dx = odefun(t,x)

dx = [
    x(2);
    F(t,x(1),x(2))
];

end
```

The same construction converts any higher-order ODE into a first-order system by introducing additional state variables.

13.10 MATLAB ODE Solvers

For a first-order system

$$\boldsymbol{y}' = \boldsymbol{f}(t, \boldsymbol{y}), \qquad \boldsymbol{y}(t_0) = \boldsymbol{y}_0,$$

MATLAB uses the general syntax

```
[t,y] = solver( ...
    odefun,tspan,y0);
```

The course lists several solvers:

```
ode45
ode23
ode113
ode23t
ode15s
ode23s
ode23tb
ode15i
```

Among these, `ode45` is presented as the standard first choice for many ordinary nonstiff problems.
The course describes it as a fourth–fifth order Runge–Kutta method.

13.11 Using `ode45`

For

$$y' = f(t, y), \qquad y(t_0) = y_0,$$

define the right-hand side as a function handle.

```
f = @(t,y) -2*y + sin(t);

[t,y] = ode45( ...
    f,[0 10],1);
```

The solution can then be plotted directly.

```
plot(t,y)
xlabel('t')
ylabel('y(t)')
grid on
```

If values are desired at specified time points, use a vector for `tspan`.

```
tspan = linspace(0,10,101);

[t,y] = ode45( ...
    f,tspan,y0);
```

13.12 Systems with `ode45`

For a system, the state is stored as a column vector.
Consider

$$\begin{cases} x' = y, \\ y' = -x. \end{cases}$$

The MATLAB function is

```
function dx = oscillator(t,x)

dx = [
    x(2);
    -x(1)
];

end
```

Then solve the system by

```
x0 = [1;0];

[t,x] = ode45( ...
    @oscillator,[0 10],x0);
```

The first and second components are stored in separate columns.

```
plot(t,x(:,1))
hold on
plot(t,x(:,2))
hold off
```

13.13 Phase-Plane Visualization

For a two-dimensional autonomous system

$$\begin{cases} x' = f(x,y), \\ y' = g(x,y), \end{cases}$$

the trajectory can be viewed either as functions of time or as a curve in the phase plane. After solving with `ode45`,

```
[t,x] = ode45( ...
    @odefun,tspan,x0);
```

the phase trajectory is

```
plot(x(:,1),x(:,2))
xlabel('x_1')
ylabel('x_2')
grid on
```

The time plot describes how each component changes, while the phase portrait describes the geometric relationship between the state variables.

13.14 Parameters in ODE Models

Many differential-equation models contain parameters.
A convenient approach is to include them in an anonymous function.

```
a = 2;
b = 3;

```

```
f = @(t,y) ...
    model(t,y,a,b);

[t,y] = ode45( ...
    f,[t0 T],y0);
```

Alternatively, the parameters may be passed through a wrapper function.
This makes it easy to repeat the experiment for different parameter values without rewriting the differential equation.

13.15 Application Pattern: Pursuit Model

The course uses a missile pursuit problem as a representative ODE modeling experiment. Suppose the target position is known as a function of time, while a pursuing object moves at prescribed speed and continuously points toward the target. The direction of its velocity therefore depends on the relative position of the two objects.
This produces a system of the form

$$\begin{cases} x' = F_1(t, x, y), \\ y' = F_2(t, x, y). \end{cases}$$

The modeling workflow is

$$\text{describe target motion} \longrightarrow \text{derive pursuit direction}$$
$$\text{construct ODE system} \longrightarrow \text{solve numerically} \longrightarrow \text{test interception.}$$

The course then uses `ode45` to compute the trajectory and checks whether interception occurs before a prescribed reaction time.
The important lesson is not the particular missile example, but the general translation

$$\text{motion rule} \longrightarrow \text{differential equation} \longrightarrow \text{numerical simulation.}$$

13.16 Online and Offline Computation

The pursuit experiment also illustrates that the same mathematical model can be used computationally in different ways.
An *online method* solves the differential equation when the parameters are supplied. It is flexible, but computation is required in real time.
An *offline method* precomputes solutions over a range of parameters and stores the resulting critical values. Real-time use then reduces to table lookup or interpolation.
A simulation method emphasizes visualization of the trajectories.
Thus mathematical experimentation also involves computational design: the numerical method should reflect how the model will ultimately be used.

13.17 Choosing an ODE Solver

For many ordinary nonstiff problems, the course recommends beginning with

```
ode45
```

For lower-accuracy calculations,

```
ode23
```

may be sufficient.
The course also lists

```
ode113
ode23t
ode15s
ode23s
ode23tb
ode15i
```

for problems requiring different numerical behavior.
In particular, when `ode45` performs poorly because the system is stiff, the course suggests trying a stiff solver such as `ode15s`.
The solver should therefore be chosen according to the structure of the differential equation rather than only by habit.

13.18 Chapter Shorthand

The principal differential-equation patterns are:

```
% Forward Euler
y(n+1) = y(n) ...
    + h*f(t(n),y(n));

% Backward Euler
F = @(z) ...
    z - y(n) ...
    - h*f(t(n+1),z);

y(n+1) = fzero(F,y(n));

% Improved Euler
yp = y(n) ...
    + h*f(t(n),y(n));

y(n+1) = y(n) ...
    + h/2*( ...
    f(t(n),y(n)) ...
    + f(t(n+1),yp));

% Classical RK4
k1 = f(tn,yn);

k2 = f( ...
    tn+h/2, ...
    yn+h*k1/2);

k3 = f( ...
    tn+h/2, ...
    yn+h*k2/2);

k4 = f( ...
    tn+h, ...
```

```
    yn+h*k3);

ynext = yn ...
    + h*( ...
    k1 + 2*k2 ...
    + 2*k3 + k4)/6;

% Built-in ODE solver
[t,y] = ode45( ...
    odefun,tspan,y0);

% Lower-order Runge--Kutta solver
[t,y] = ode23( ...
    odefun,tspan,y0);

% Stiff solver
[t,y] = ode15s( ...
    odefun,tspan,y0);

% First-order system
function dx = odefun(t,x)

dx = [
    equation1;
    equation2
];

end

% Time-series plot
plot(t,y)

% Phase-plane plot
plot(y(:,1),y(:,2))
axis equal
grid on
```

The central idea is to replace continuous evolution by a sequence of discrete updates. Forward Euler gives the simplest explicit approximation, implicit methods improve stability, and Runge–Kutta methods improve accuracy by combining several slope evaluations. For practical computation, MATLAB ODE solvers automate step-size and error control, while the main mathematical task remains the construction of the differential-equation model and the interpretation of its numerical solution.

Acknowledgements

I would like to express my sincere gratitude to my advisor, Professor Kailiang Wu, who provided the lecture slides on which these notes are based and also taught the course *Mathematical Experiments*.
In his lectures, Professor Wu did not present MATLAB merely as a collection of computational tools. He frequently introduced examples from everyday life and practical problems to illustrate why mathematical experiments are useful: computation can reveal patterns, test ideas, visualize structures, and help us understand mathematical models that arise from real situations.
This perspective also influenced the way I learned the course. Rather than viewing mathematics only as a system of established definitions, theorems, and techniques, I gradually learned to approach it from the perspective of *discovering mathematics*: observing numerical phenomena, formulating conjectures, experimenting with models, and then returning to mathematical reasoning to understand why the observed patterns occur.
These notes are therefore not intended merely as a MATLAB command reference. They also record a way of thinking about mathematics through computation, experimentation, observation, and discovery.